

深圳大学实验报告

课程名称 计算机游戏开发

项目名称 实验 3 二维游戏动画合成

学 院 数学科学学院

专 业 信息与计算科学（数学与计算机实验班）

指导教师 储 颖

报 告 人 王曦 学号 2021192010

实验时间 2024 年 05 月 03 日

提交时间 2024 年 05 月 03 日

教务处制

目录

目录.....	2
一、实验目的与要求.....	4
二、实验内容与方法.....	4
三、实验步骤与过程.....	5
1. 重构与运行.....	5
1.1 重构.....	5
1.2 编译运行.....	6
2. 计分板与多重关卡.....	8
2.1 计分板.....	8
2.2 多重关卡.....	11
3. 增加 Hero 的 Defend 动作.....	12
3.1 绘制 Defend 动作.....	12
3.2 加载 Defend 动作.....	17
4. 修复 Bug.....	21
4.1 Hero 受伤后无法操控.....	21
4.2 攻击按钮也控制了 Tauren.....	22
4.3 多次掉血累加.....	22
4.4 Tauren 左倾.....	23
4.5 角色的移动超出世界边界.....	24
4.6 Tauren 的攻击区域过小.....	26
4.7 碰撞检测不严谨.....	28
4.8 连续点击技能.....	29
4.9 smitten 动画未执行.....	30
4.10 attack 动画会被 smitten 动画打断.....	30
4.11 云朵移动动画衔接不上.....	30
4.12 角色朝向切换.....	31
4.13 Hero 诈尸.....	32
4.14 攻击、受伤后无法恢复 loading 动作.....	33
4.15 Hero 很靠近 Tauren 时持续受伤.....	34
4.16 角色受伤时攻击被打断.....	35
5. 游戏音乐.....	36
5.1 加载与播放音乐的函数.....	36
5.2 关卡音乐的选择.....	37
5.3 加载关卡 1 的音乐.....	38
6. 游戏优化.....	41
6.1 游戏策划.....	41
6.2 Tauren AI 优化.....	42
6.3 随机性优化.....	45
6.4 开始界面.....	47
6.5 独立计分板.....	52
6.6 第一关.....	52
6.7 第二关.....	54

6.8 第三关.....	59
6.9 通关界面.....	66
四、实验结论或心得体会.....	67
7. 实验结论.....	67
7.1 编译运行.....	67
7.2 增加计分板与多重关卡.....	67
7.3 增加 Hero 的 Defend 动作.....	68
7.4 修复 Bug.....	69
7.5 增加配乐和音效.....	69
7.6 游戏优化.....	70
8. 实验心得.....	71

一、实验目的与要求

- 1.了解二维游戏动画合成原理。
- 2.熟悉 Cocos2d-x 中的用户交互、触摸事件、碰撞检测机制。
- 3.熟悉 CocoStudio 动画编辑器的使用，了解骨骼动画。

二、实验内容与方法

1. 完成游戏编译（5 分）

成功编译并运行教材 P128 “游戏动画实例-侠客行”。

2. 增加计分板功能和多重关卡（10 分）

记录成功/失败次数，增加计分板功能；将游戏改为多重关卡游戏。

3. 增加英雄 Defend 动作（15 分）

利用 CocoStudio 设计英雄 Defend 动作，并将此功能在游戏中加载。

4. 修改游戏 bug（15 分）

修改游戏中出现的明显 bug。

5. 增加配乐和音效（5 分）

为游戏添加合适的配乐和音效。

5. 游戏优化升级（10 分）

自行发挥想象力，优化游戏功能。

6. 完成实验报告（40 分）

截图记录关键步骤，分析实验结果，撰写心得体会。

三、实验步骤与过程

(记录关键步骤/设计过程/设计结果的截图)

1. 重构与运行

1.1 重构

将本次实验的示例代码重构，并替换部分美术资源。

1.1.1 工程结构

重构后的工程结构如下：

- (1) AppDelegate.h、AppDelegate.cpp
- (2) Headers.h: 引用工程所需的头文件和命名空间。
- (3) Config.h、Config.cpp: 定义了角色状态、资源路径、游戏参数，并实现了用于比较浮点数大小的函数 `doubleCompare()` 和随机生成区间 `[l, r]` 内的整数的函数 `generateRandomInteger()`。
- (4) BasicScene.h、BasicScene.cpp: 场景类基类，实现了添加键盘监听、加载图片、预加载音乐、播放/暂停/恢复音乐、显示提示信息、游戏胜利、游戏失败等函数。
- (5) Joystick.h、Joystick.cpp: **实验 2** 中的摇杆类。
- (6) BloodBar.h、BloodBar.cpp: 血条类，实现血条的显示、更新。
- (7) Hero.h、Hero.cpp: 英雄类，定义了 Hero 的状态、血量等属性，实现了创建、切换状态、播放帧动画、受伤、显示扣血等函数。
- (8) Tauren.h、Tauren.cpp: 牛头人类，定义了 Tauren 的状态、血量等属性，实现了创建、切换状态、播放帧动画、受伤、显示扣血等函数。
- (9) TaurenAI.h、TaurenAI.cpp: 牛头人 AI 类，控制 Tauren 的行为。
- (10) CollisionDetection.h、CollisionDetection.cpp: 碰撞检测类，检测并处理 Hero 与 Tauren 的碰撞。
- (11) Level1Scene.h、Level1Scene.cpp: 关卡 1 类，继承于 BasicScene 类，实现 Hero 和 Tauren 的加载、控制、血条更新、计分板和 Hero 的攻击等函数。

部分类的具体实现见 **6. 游戏优化**。

1.1.2 资源文件结构

将 Resources 文件夹中的美术资源分类，并替换了部分美术资源，其中已将 `cloud.png` 抠图。如图 1.1.2.1 所示。

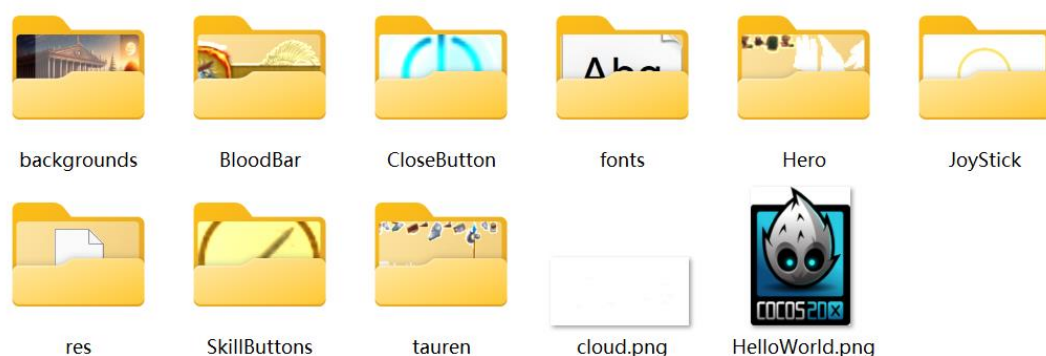


图 1.1.2.1: 将 Resources 文件夹中的美术资源分类

1.2 编译运行

1.2.1 构建项目

在 `cocos2d-x-3.17.2\tools\cocos2d-console\bin` 目录下，打开命令行，用命令

```
cocos new -l cpp EXP3
```

构建项目，并替换 Classes 文件夹和 Resources 文件夹。如图 1.2.1.1 所示。

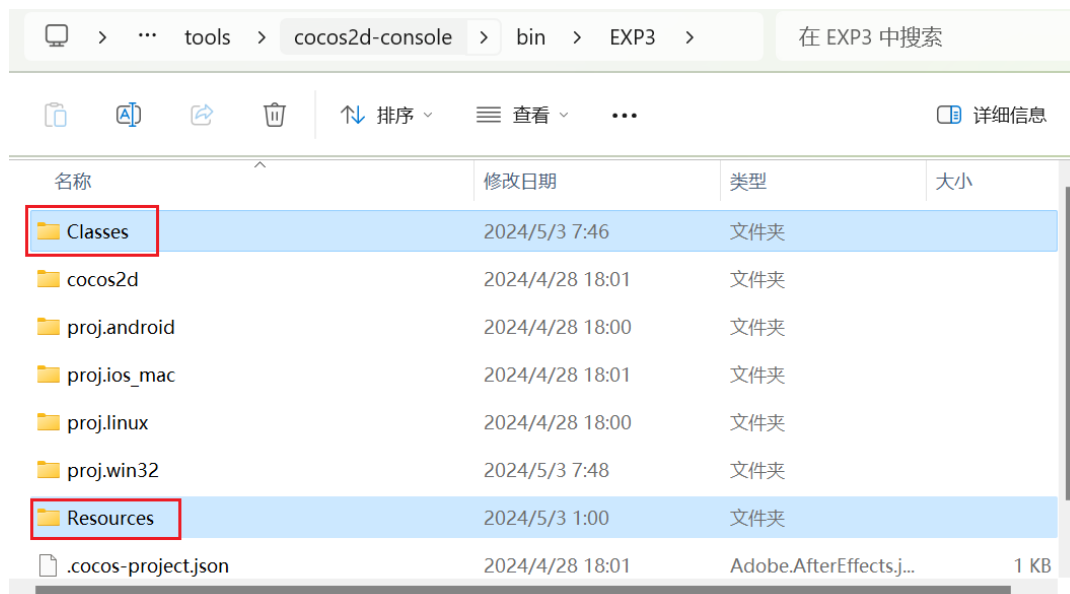


图 1.2.1.1: 替换 Classes 文件夹和 Resources 文件夹

在 Visual Studio 中将 Classes 文件夹中的文件都添加到 src 下。如图 1.2.1.2 所示。

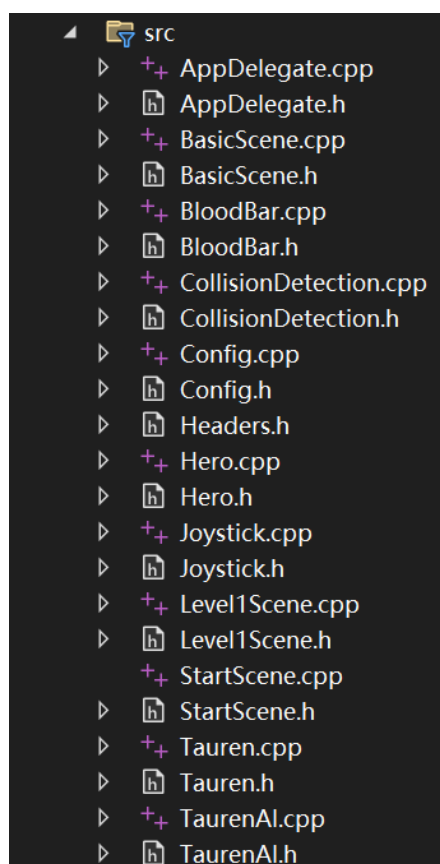


图 1.2.1.2: 将 Classes 文件夹中的文件都添加到 src 下

1.2.2 编译运行

成功编译并运行《侠客行》。如图 1.2.2.1 所示。

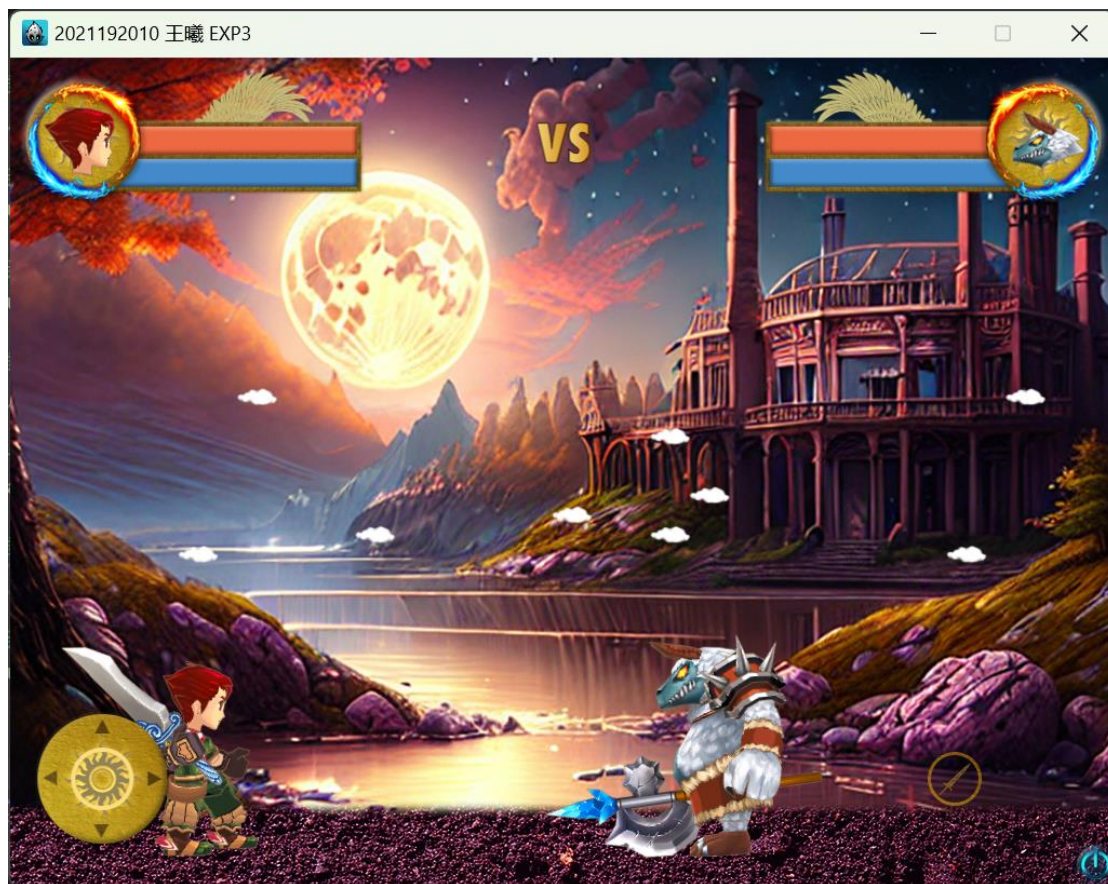


图 1.2.2.1：成功编译并运行《侠客行》

2. 计分板与多重关卡

2.1 计分板

《侠客行》设多个关卡，玩家可多次尝试同一关。增加计分板，每关结束后，胜者得 1 分。

本次实验仅实现了一个朴素的计分板，故未将其独立成类，而将其作为 Level1Scene 的一部分。

2.1.1 创建计分板

为 Level1Scene 添加 Hero 和 Tauren 的得分和计分板 (Label)。

```
// 计分板
int heroScore;
Label* heroScoreboard;
int taurenScore;
Label* taurenScoreboard;
```

声明并实现了加载 Hero 计分板的函数 loadHeroScoreboard()和加载 Tauren 计分板的函数 loadTaurenScoreboard()。

```
// 加载 Hero 计分板
void Level1Scene::loadHeroScoreboard(int score, Point position, int z, int tag) {
    heroScore = score;
    heroScoreboard = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(heroScore));
    heroScoreboard->setPosition(position);
    if (~tag) {
        heroScoreboard->setTag(tag);
    }
    this->addChild(heroScoreboard, z);
}

// 加载 Tauren 计分板
void Level1Scene::loadTaurenScoreboard(int score, Point position, int z, int tag) {
    taurenScore = score;
    taurenScoreboard = Label::createWithBMFont(TIPS_FNT_PATH,
std::to_string(taurenScore));
    taurenScoreboard->setPosition(position);
    if (~tag) {
        taurenScoreboard->setTag(tag);
    }
    this->addChild(taurenScoreboard, z);
}
```

在 Level1Scene 的 init()函数中加载即可。

```
// 计分板
heroScore = 0;
loadHeroScoreboard(0, Point(visibleSize.width / 2 - 100, visibleSize.height - 80), 5);
taurenScore = 0;
loadTaurenScoreboard(0, Point(visibleSize.width / 2 + 100, visibleSize.height - 80), 5);
```


运行结果如图 2.1.1.1 所示。

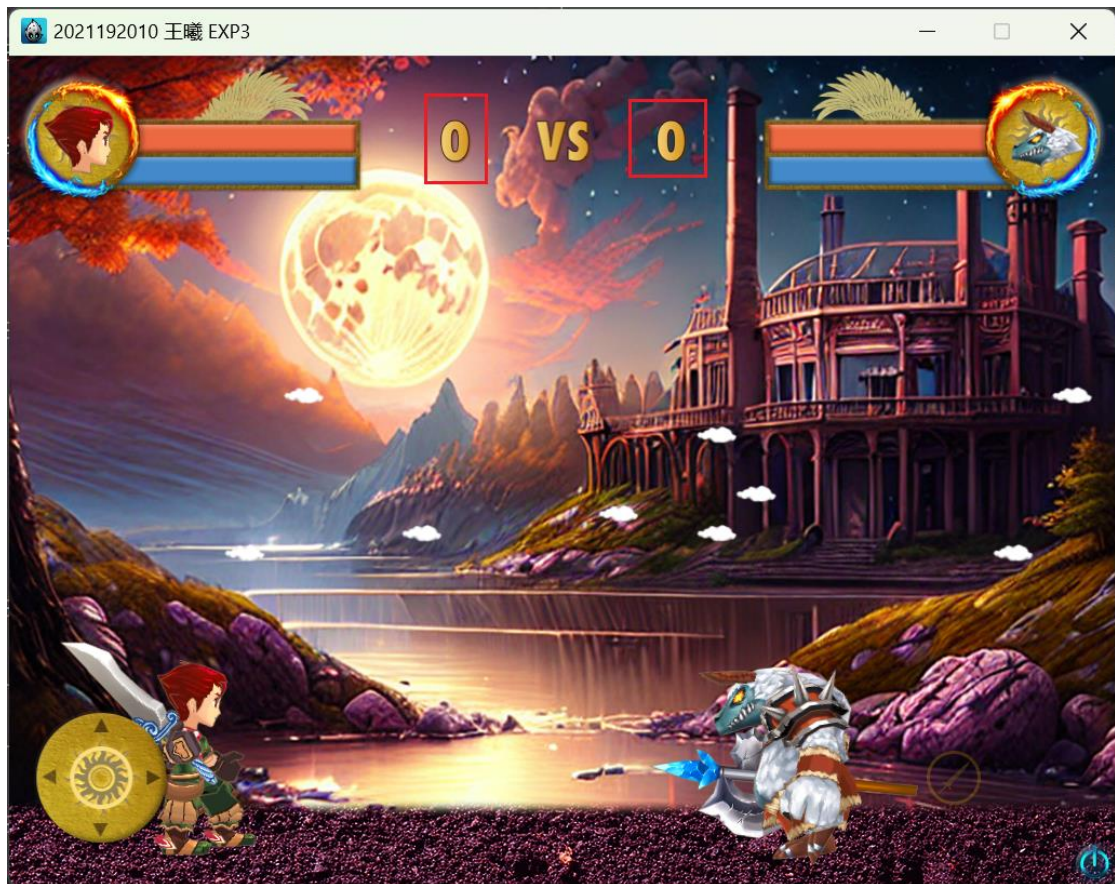


图 2.1.1.1: 创建计分板

2.1.2 更新计分板

Level1Scene 类重写父类 BasicScene 中游戏胜利函数 workVictory()、游戏失败函数 workDefeat()和游戏胜负判定函数 gameJudgement()。

```
// 游戏胜利
void Level1Scene::workVictory() {
    pause = true;

    taurenAI->stopAllActions();

    updateHeroScoreboard(heroScore + 1);
}

// 游戏失败
void Level1Scene::workDefeat() {
    pause = true;

    taurenAI->stopAllActions();

    updateTaurenScoreboard(taurenScore + 1);
}
```

```
// 游戏胜负判定
void Level1Scene::gameJudgement() {
    // 胜负判定
    if (doubleCompare(hero->getHP(), 0) <= 0) {
        workDefeat();
    }
    else if (doubleCompare(tauren->getHP(), 0) <= 0) {
        workVictory();
    }
}
```

其中游戏胜利、失败时调用 `updateHeroScoreboard()` 函数或 `updateTaurenScoreboard()` 函数更新胜者的计分板，实现方式：先删除旧标签，然后更新分数，再创建新标签。

```
// 更新 Hero 计分板
void Level1Scene::updateHeroScoreboard(int score) {
    Point position = heroScoreboard->getPosition();
    int z = heroScoreboard->getZOrder();
    int tag = heroScoreboard->getTag();
    this->removeChild(heroScoreboard);

    heroScore = score;
    heroScoreboard = Label::createWithBMFont(TIPS_FNT_PATH, std::to_string(heroScore));
    heroScoreboard->setPosition(position);
    if (~tag) {
        heroScoreboard->setTag(tag);
    }
    this->addChild(heroScoreboard, z);
}
```

```
// 更新 Tauren 计分板
void Level1Scene::updateTaurenScoreboard(int score) {
    Point position = taurenScoreboard->getPosition();
    int z = taurenScoreboard->getZOrder();
    int tag = taurenScoreboard->getTag();
    this->removeChild(taurenScoreboard);

    taurenScore = score;
    taurenScoreboard = Label::createWithBMFont(TIPS_FNT_PATH,
std::to_string(taurenScore));
    taurenScoreboard->setPosition(position);
    if (~tag) {
        taurenScoreboard->setTag(tag);
    }
    this->addChild(taurenScoreboard, z);
}
```

}

以 Hero 打败 Tauren 为例，运行结果如图 2.1.2.1 所示，Hero 得 1 分。

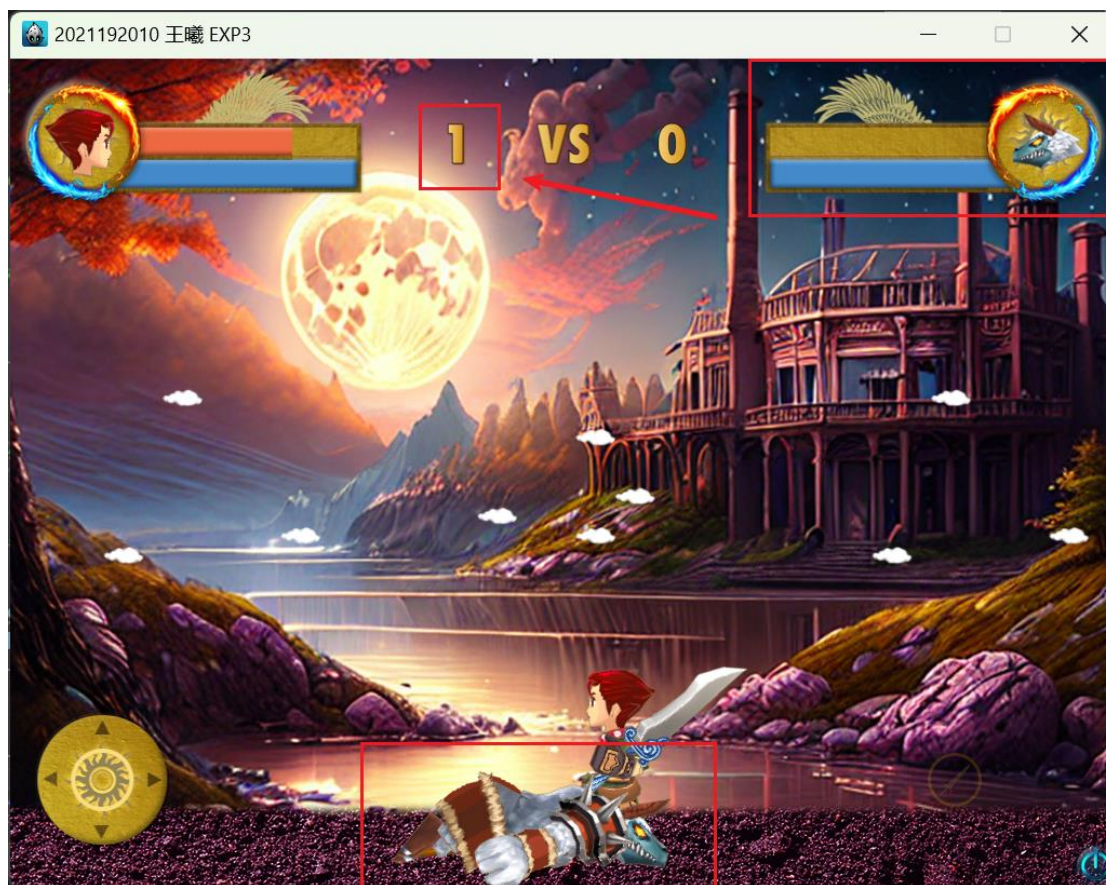


图 2.1.2.1: Hero 得 1 分

2.2 多重关卡

见 6.1.5 关卡设置。

3. 增加 Hero 的 Defend 动作

3.1 绘制 Defend 动作

3.1.1 导入并绑定盾牌

绘制一个 80 x 80 的盾牌，如图 3.1.1.1 所示。



HeroShield.png

图 3.1.1.1: 盾牌

在 Cocos Studio 的 Animation Editor 中导入盾牌。如图 3.1.1.2 所示。

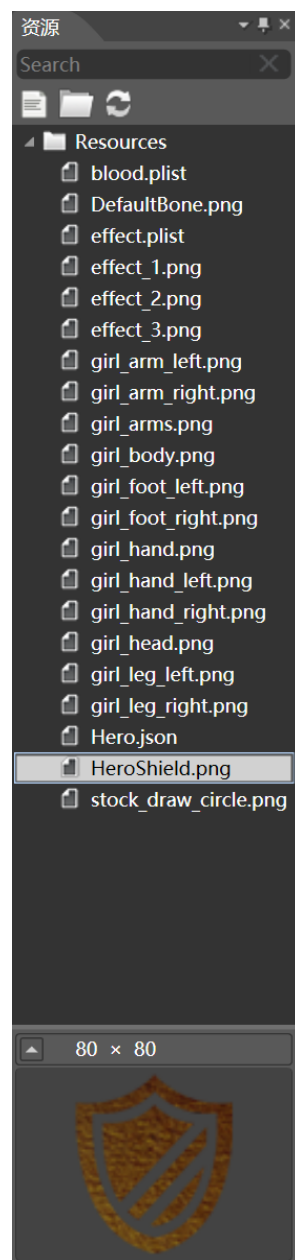


图 3.1.1.2: 在 Animation Editor 中导入盾牌

将盾牌拖入形体模式的画布中，调整到合适的位置和角度，沿手持方向绘制骨骼。右键盾牌，选择“绑定到骨头”，左键点击新增的骨骼。右键新增的骨骼，选择“绑定父关系”，左键点击 Hero 的左手的骨骼。至此完成盾牌的绑定，如图 3.1.1.3 所示。



图 3.1.1.3: 导入并绑定盾牌

3.1.2 补充已有动画

切换到动画模式，发现除了 loading 动作外，其它动作都无盾牌。如图 3.1.2.1 所示。

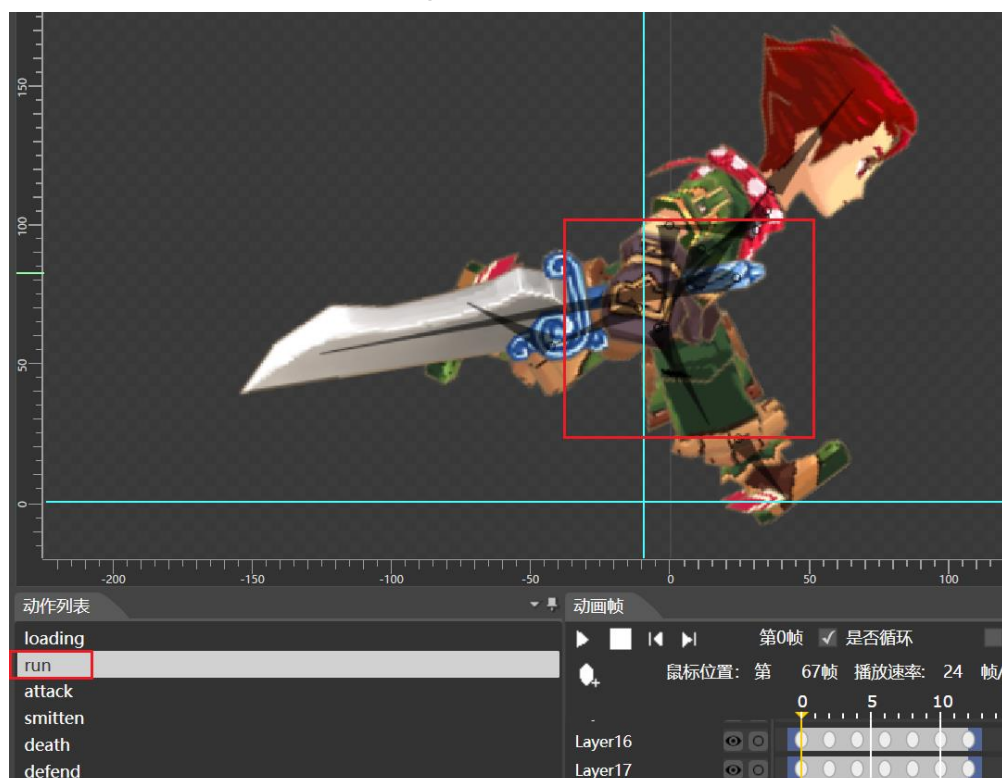


图 3.1.2.1: run 动作无盾牌

在动画帧栏中盾牌所在层（Layer6）的第 0 帧添加帧即可。后续帧盾牌将随左手的运动

而运动。如图 3.1.2.2 所示。

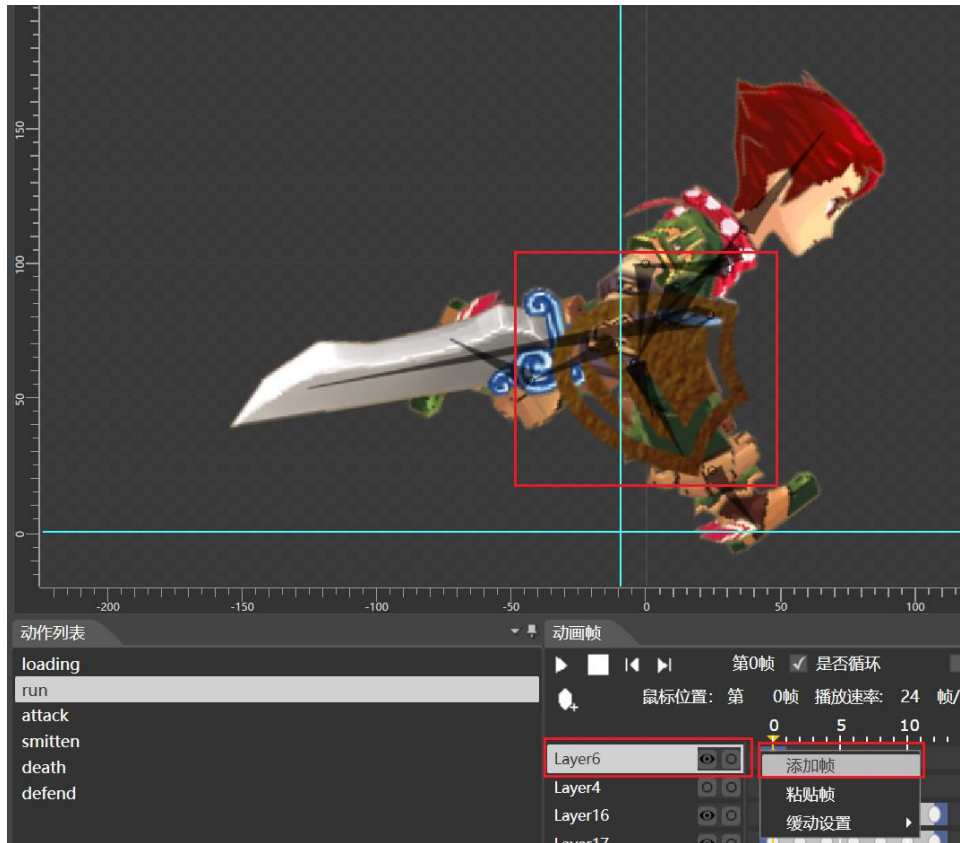


图 3.1.2.2：在盾牌所在层的第 0 帧添加帧

同理为 attack、smitten、death 动画添加盾牌的 Layer 的帧。

至此，已有的动画都已有盾牌。

3.1.3 绘制 Defend 动画

在动画列表中右键，选择“添加动画”，再右键新建的动画，选择“重命名 F2”，将其名称修改为“defend”。如图 3.1.3.1 所示。

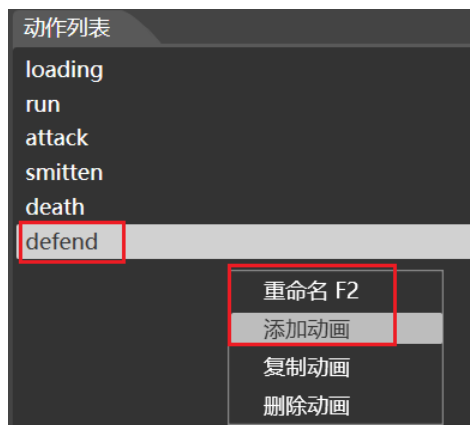


图 3.1.3.1：新建 defend 动画

在动画帧中，为除 Layer3 和 Layer4 外的层的第 0 帧添加帧，并修改播放速率为 24 帧/秒。如图 3.1.3.2 所示。

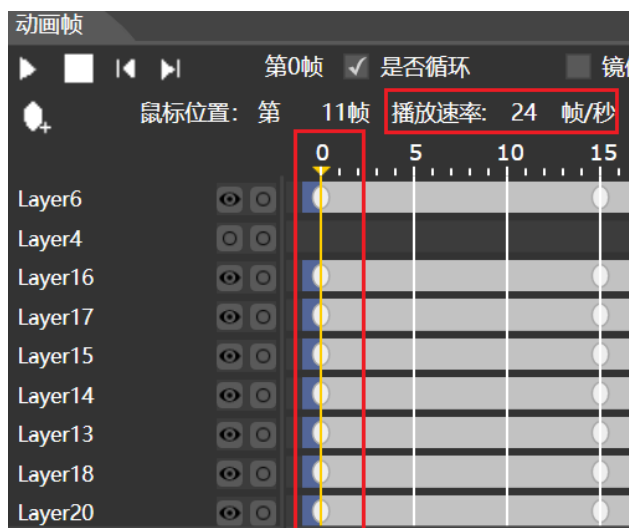


图 3.1.3.2: 为除 Layer3 和 Layer4 外的层的第 0 帧添加帧
 为上述层在第 15 帧、30 帧的位置分别添加帧，其中第 15 帧作为 defend 动作的最终形态，第 30 帧为恢复初始状态。如图 3.1.3.3 所示。

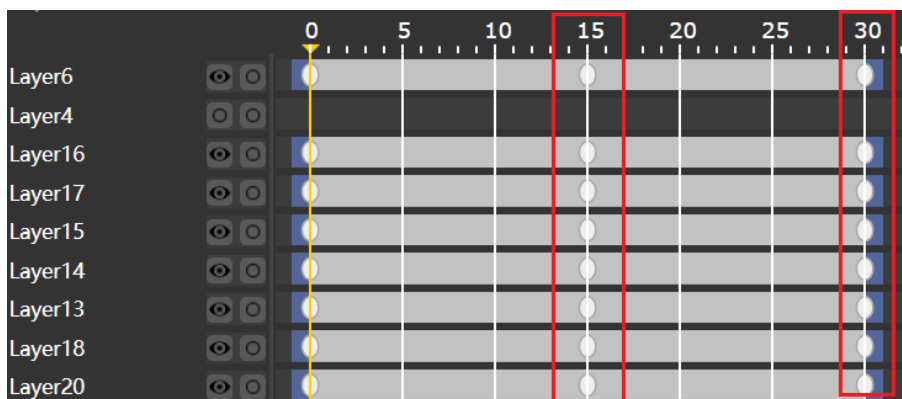


图 3.1.3.3: 在第 15 帧、30 帧的位置分别添加帧
 选择第 0 帧，拉参考线。如图 3.1.3.4 所示。



图 3.1.3.4: 在第 0 帧拉参考线
 选择第 15 帧，调整出 defend 动作的姿势。具体地：

- (1) 左腿往前迈一步，类似于弓步。
- (2) 手举起盾牌。
- (3) 头往上扬。

如图 3.1.3.5 所示。

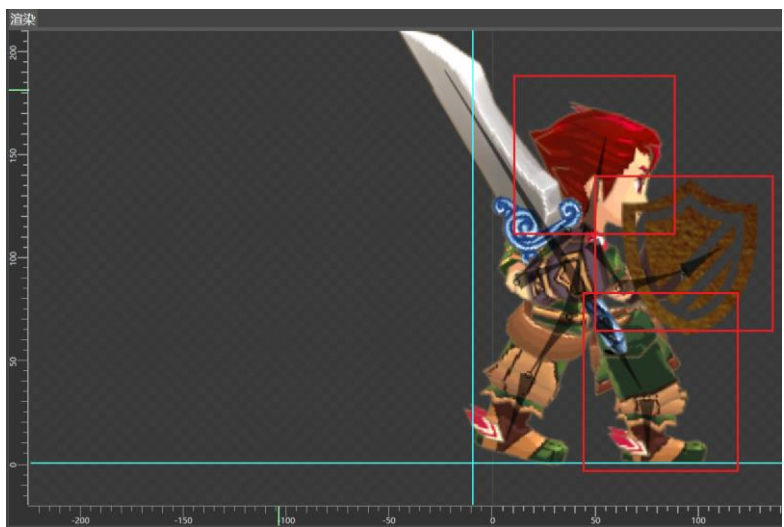
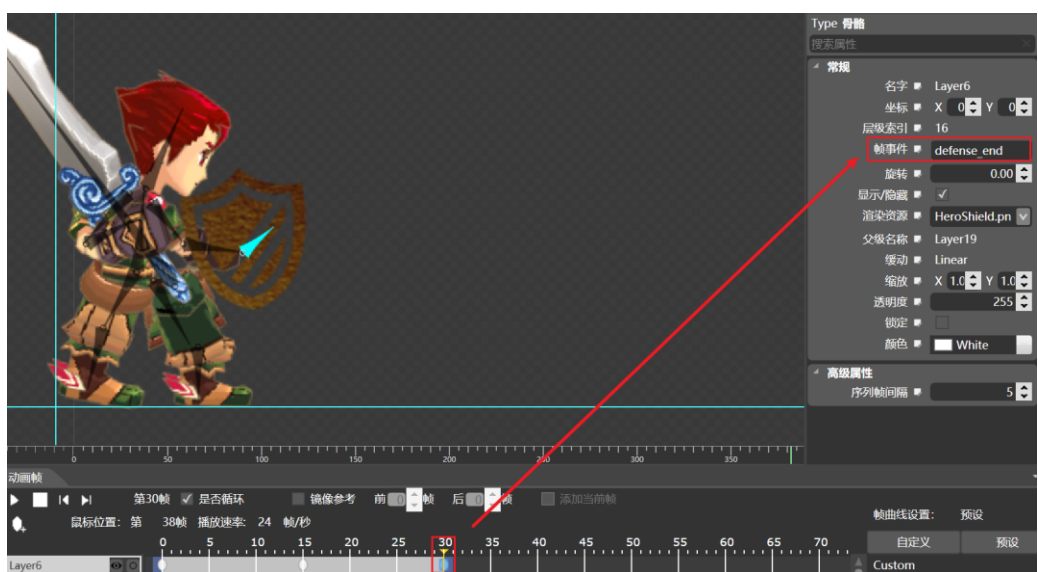


图 3.1.3.5: Defend 动作

选择 defend 动作任意层的最后一帧，为其添加帧事件 “defense_end”。



Ctrl + S 保存项目，Ctrl + E 导出项目，选择“导出使用大图”。如图 3.1.3.6 所示。

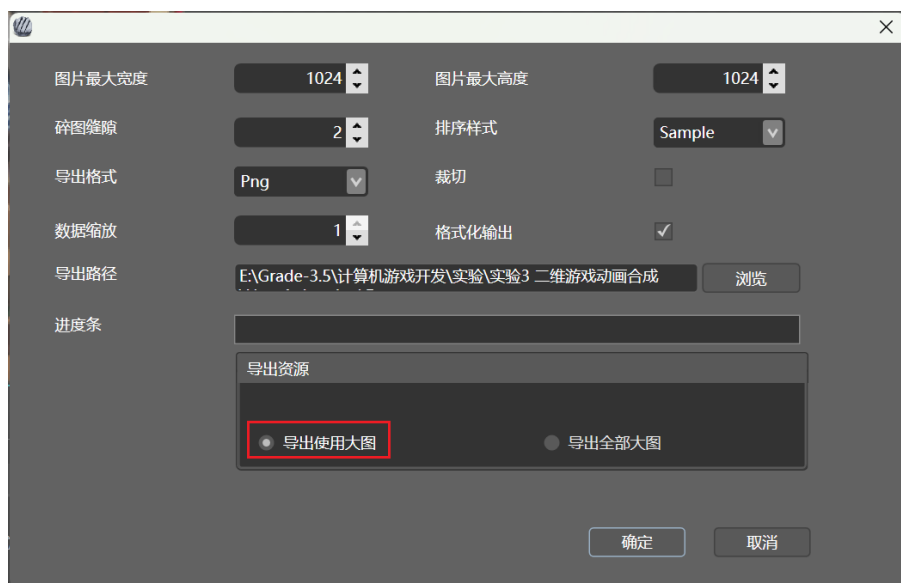


图 3.1.3.6: 导出项目，选择“导出使用大图”

将导出结果复制到 Resources/Hero 文件夹下。如图 3.1.3.7 所示。

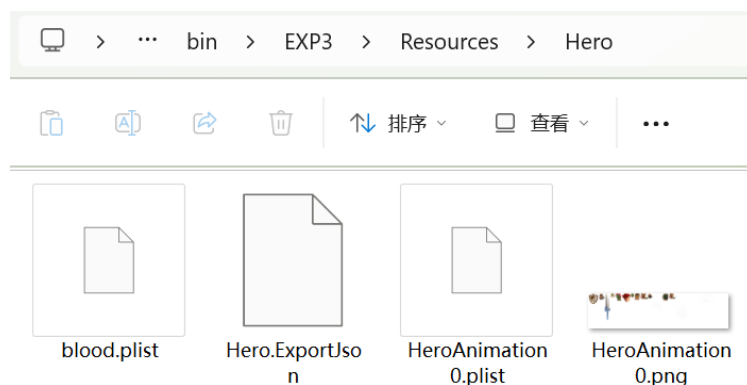


图 3.1.3.7: 将导出结果复制到 Resources/Hero 文件夹下

3.2 加载 Defend 动作

3.2.1 添加 Defend 按钮

在 Config.h 中为角色状态的枚举类型 State 添加 DEFENSE 状态。

```
// 角色状态
```

```
enum State {
    STAND,
    MOVELEFT,
    MOVERIGHT,
    ATTACK,
    DEAD,
    SMITTEN,
    DEFENSE
};
```

类似于攻击按钮，在 Level1Scene 中添加加载防御按钮的函数 loadDefenseButton()和防御按钮的回调函数 defenseButtonCallback()。

```
// 加载防御按钮
```

```

void Level1Scene::loadDefenseButton(Point position, int z, int tag = -1) {
    auto defenseImage = MenuItemImage::create(
        DEFENSE_BUTTON, DEFENSE_BUTTON_PRESSED,
        CC_CALLBACK_1(Level1Scene::defenseButtonCallback, this)
    );
    defenseImage->setPosition(position);

    // 闪烁
    auto actionSequence = Sequence::create(
        FadeOut::create(1),
        FadeIn::create(1),
        nullptr
    );
    auto action = RepeatForever::create(actionSequence);
    defenseImage->runAction(action);

    auto defenseButton = Menu::create(defenseImage, nullptr);
    defenseButton->setPosition(Point::ZERO);
    if (~tag) {
        defenseButton->setTag(tag);
    }

    this->addChild(defenseButton, z);
}

// 防御按钮的回调函数
void Level1Scene::defenseButtonCallback(Ref* pSender) {
    hero->switchState(State::DEFENSE);
}

```

在 Level1Scene 的 init()函数中加载。

```
loadDefenseButton(Point(visibleSize.width - 80, 100), 5);
```

运行结果如图 3.2.1.1 所示。

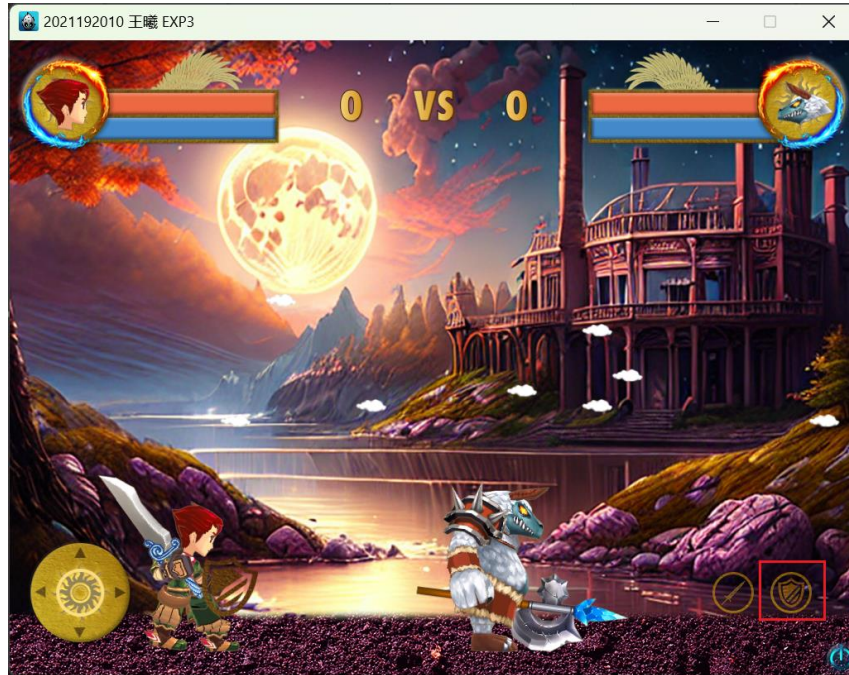


图 3.2.1.1: 防御按钮

3.2.2 实现 Defend 动作

在 Hero.h 中为 Hero 类添加表示是否处于防御状态的成员变量 isDefensing。

在 Hero 类的 init()函数中初始化。

```
isDefensing = false;
```

在 Hero 类的 update()函数的 switch 中加入 DEFENSE 状态的分支，播放动画并设置 isDefensing = true。

```
void Hero::update(float dt) {
    ...

    switch (state) {
        ...
        case DEFENSE: {
            if (!isDead) {
                armature->getAnimation()->play("defend");
                isDefensing = true;
            }

            break;
        }
    }
}
```

defend 动画结束时恢复 loading 状态。

```
// 帧动画
void Hero::onFrameEvent(cocostudio::Bone* bone, const std::string& evt, int originFrameIndex,
int currentFrameIndex) {
    ...
}
```

```

else if (evt == "defense_end") {
    armature->getAnimation()->play("loading");
    isDefensing = false;
}
}

```

本次实验设置的 Defend 动作只是在受伤时抵消一部分伤害,即 Hero 被攻击时仍要受伤,这导致 defend 动作可能会被 smitten 动作打断,即 defend 动作可能未播放到 defense_end 就被终止,这将导致 isDefensing 无法复位。为解决该问题,在受伤时令 isDefensing = false。

```

// 切换状态
void Hero::switchState(State _state) {
    if (_state == SMITTEN) {
        armature->getAnimation()->play("smitten");
        isHurt = true;
        isDefensing = false;
    }

    state = _state;
}

```

Hero 受伤时,若有 isDefensing,则抵消部分伤害。

```

// 受伤
void Hero::playHurt() {
    if (isDefensing) {
        displayBloodTip(std::max(BASIC_DAMAGE - DEFENSE_DAMAGE, 0));
    }
    else {
        displayBloodTip(BASIC_DAMAGE);
    }
    this->switchState(SMITTEN);
}

```

其中基础伤害和防御抵消伤害在 Config.h 中设置。

```

static const int BASIC_DAMAGE = 10; // 基础攻击力
static const int DEFENSE_DAMAGE = 5; // 防御抵消的攻击力

```

3.2.3 运行结果

有无防御受伤时的掉血如图 3.2.3.1 和图 3.2.3.2 所示。



图 3.2.3.1: 无防御受伤



图 3.2.3.2: 有防御受伤

运行结果见【附件】侠客行。

4. 修复 Bug

本部分的运行结果见【附件】侠客行。

4.1 Hero 受伤后无法操控

Hero 受伤后卡在原地，无法操控。

检查发现，Hero 类的 update() 函数中，切换为 STAND、MOVELEFT、MOVERIGHT 状态需要要求 !isHurt，故 Hero 受伤后无法切换到这 3 个状态。然而，切换到这 3 个状态要求 !isHurt 是不合理的，将其删去即可。

```
void Hero::update(float dt) {
    ...

    switch (state) {
        case STAND: {
            // 跑动结束只执行一次 loading，避免点击 attack 动作无法播放
            // if (!isDead && isRunning && !isHurt) {
            if (!isDead && isRunning) {
                ...
            }

            break;
        }
        case MOVELEFT: {
            // if (!isDead && this->getPositionX() > 0 && !isHurt && !isAttacking) {
            if (!isDead && this->getPositionX() > 0 && !isAttacking) {
                ...
            }

            break;
        }
        case MOVERIGHT: {
            // if (!isDead && this->getPositionX() <
            if (!isDead && this->getPositionX() <
            Director::getInstance()->getVisibleSize().width && !isHurt && !isAttacking) {
            Director::getInstance()->getVisibleSize().width && !isAttacking) {
                ...
            }

            break;
        }
        ...
    }
}
```

修改后，Hero 受伤后玩家仍能通过摇杆操控。

4.2 攻击按钮也控制了 Tauren

玩家点击攻击按钮后，Hero 和 Tauren 都攻击。如图 4.2.1 所示。



图 4.2.1：玩家点击攻击按钮后，Hero 和 Tauren 都攻击

检查发现，Level1Scene 中攻击按钮的回调函数 attackButtonCallback()也切换了 Tauren 的状态，将其删去即可。

```
// 攻击按钮的回调函数
void Level1Scene::attackButtonCallback(Ref* pSender) {
    hero->switchState(State::ATTACK);
    // tauren->switchState(State::ATTACK);
}
```

修改后，玩家点击攻击按钮只能操控 Hero 攻击。

4.3 多次掉血累加

Hero 被攻击时，有时会多次掉血。如图 4.3.1 所示。



图 4.3.1：Hero 被攻击时多次掉血

事实上这未必是 Bug，可能是原作者希望实现有一定概率发生连击。若不希望出现连击，可在 Hero 类的 displayBloodTip()函数中保持连击次数 hitCount = 1。

```
// 显示扣血
void Hero::displayBloodTip(int basicDamage) {
    // 连击
```



```
int hitCount = 1; // 连击次数
int hitRand = generateRandomInteger(0, 9);
if (3 < hitRand && hitRand < 8) {
    hitCount = 2;
}
else if (hitRand > 7) {
    hitCount = 3;
}

hitCount = 1; // 取消连击

...
}
```

修改后，Hero 每次被攻击只会掉血一次。

4.4 Tauren 左倾

Tauren 的 AI 虽然偶尔会回头，但明显倾向于向左走，即使玩家在右侧，它也执着地走到左边面壁。如图 4.4.1 所示。



图 4.4.1: Tauren 的 AI 明显倾向于向左走

检查发现，TaurenAI 的动作序列的每个周期中，向左走的时间占 1.0 s，向右走的时间占 0.7 s，故 Tauren 有严重的左倾倾向。

增大向右走的时间即可。具体地，有如下 3 种情况：

(1) 向右走的时间 < 向左走的时间：仍有左倾倾向，但增大向右走的时间可以降低一些左倾倾向。

(2) 向右走的时间 = 向左走的时间：无倾向，动作不被打断时，会向左走到一定位置后，再向右走回原位置。

(3) 向右走的时间 > 向左走的时间：有右倾倾向，增大向右走的时间可以提高一些右

倾倾向。

本次实验以情况（1）为例，取向右走的时间为 0.85 s。

```
void TaurenAI::setAI(Hero* _hero, Tauren* _tauren) {  
    ...  
  
    auto sequence = Sequence::create(  
        DelayTime::create(0.8f),  
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveLeft, this)),  
        DelayTime::create(1.0f),  
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),  
        DelayTime::create(0.3f),  
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveRight, this)),  
        // DelayTime::create(0.7f),  
        DelayTime::create(0.85f),  
        CallFunc::create(CC_CALLBACK_0(TaurenAI::stand, this)),  
        DelayTime::create(0.5f),  
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),  
        nullptr  
    );  
    ...  
}
```

修改后，Tauren 主要往左走，但也会往右走来一个回马枪。

但事实上，这种 AI 仍是劣质的，因为它没有考虑 Tauren 与 Hero 的相对位置，玩家可以像图 4.4.1 一样躲在右侧放远程技能。

TaurenAI 的优化见 6.2 Tauren AI 优化。

4.5 角色的移动超出世界边界

Hero 和 Tauren 的移动可能超出世界边界。如图 4.5.1 所示。



图 4.5.1: Hero 的移动超出世界边界

检查发现，Level1Scene 类中控制 Hero 移动的函数 workHeroMovement() 未限制 Hero 的 x 坐标的范围，将其修改为根据摇杆力度设置移动距离，并将目标点的 x 坐标限制在一定范围内。

```
// 控制 Hero 移动  
void Level1Scene::workHeroMovement() {  
    Vec2 direction = joystick->getDirection();
```

```

float dx = direction.x;
int sgn = doubleCompare(dx, 0);
if (!sgn) {
    hero->switchState(STAND);
}
else if (sgn < 0) {
    hero->switchState(MOVELEFT);
    // hero->setPositionX(hero->getPositionX() - SPEED_X);
}
else {
    hero->switchState(MOVERIGHT);
    // hero->setPositionX(hero->getPositionX() + SPEED_X);
}

Point destination = hero->getPosition() + Vec2(direction.x, 0) * joystick->getVelocity() * 0.1;
// 预期目标点
destination.x = std::max(destination.x, heroWidth / 2),
    destination.x = std::min(destination.x, visibleSize.width - heroWidth / 2);
hero->setPosition(destination);
}

```

类似地，修改 Tauren 类的 update()函数中的左移和右移。

```

void Tauren::update(float dt) {
    ...

    switch (state) {
        ...
        case MOVELEFT: {
            if (!isDead && this->getPositionX() > 0 && !isHurt && !isAttacking) {
                ...

                float targetX = this->getPositionX() - SPEED_X;
                targetX = std::max(targetX, 180.0f);
                this->setPositionX(targetX);
            }

            break;
        }
        case MOVERIGHT: {
            if (!isDead && this->getPositionX() <
                Director::getInstance()->getVisibleSize().width && !isHurt && !isAttacking) {
                ...

                float targetX = this->getPositionX() + SPEED_X;
                targetX = std::min(targetX, Director::getInstance()->getVisibleSize().width -

```

```

180.0f / 2);

        this->setPositionX(targetX);
    }

    break;
}
...
}
}

```

其中 heroWidth 和 taurenWidth 是根据骨骼动画指定的大致宽度,本次实验中取 heroWidth = 100, taurenWidth = 180。

修改后, Hero 和 Tauren 的移动都不会超出世界边界。

4.6 Tauren 的攻击区域过小

Tauren 的攻击区域过小,有时斧头的贴图都将玩家插穿了,都认为未命中玩家。如图 4.6.1 所示。



图 4.6.1: Tauren 的攻击区域过小

检查发现, CollisionDetection 类的 update()函数中对 Tauren 的斧头的取点过少,新增一关键点即可。

```

void CollisionDetection::update(float dt) {
    ...

    // Tauren 攻击 Hero
    Point taurenPoint1 =
tauren->getArmature()->getBone("ax")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
0));
    Point taurenPoint2 =
tauren->getArmature()->getBone("ax")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
30));
    Point taurenPoint3 =
tauren->getArmature()->getBone("ax")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
70)); // 新增关键点
    std::vector<Point> taurenPoints = { taurenPoint1, taurenPoint2, taurenPoint3 };

    if (!hero->getIsDead() && tauren->getIsAttacking()) {
        Rect heroRect = Rect(hero->getPositionX() + 10, hero->getPositionY() + 10, 20, 40);
    }
}

```

```

    for (auto& taurenPoint : taurenPoints) {
        if (heroRect.containsPoint(taurenPoint)) {
            hero->playHurt();
            tauren->setIsAttacking(false);
            // taurenAI->setTaurenState(State::STAND); // ?
            break;
        }
    }
}

```

修改后，Tauren 与 Hero 相距如图 4.6.2 所示时已可攻击到。



图 4.6.2: Tauren 的最大攻击距离（图中为命中后 Tauren 转身）

同理提高 Hero 的最大攻击距离。

```

void CollisionDetection::update(float dt) {
    // Hero 攻击 Tauren
    Point heroPoint1 =
hero->getArmature()->getBone("Layer17")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
0));
    Point heroPoint2 =
hero->getArmature()->getBone("Layer17")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
15));
    Point heroPoint3 =
hero->getArmature()->getBone("Layer17")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
30));
    Point heroPoint4 =
hero->getArmature()->getBone("Layer17")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
50));
    Point heroPoint5 =
hero->getArmature()->getBone("Layer17")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
70)); // 新增关键点
    std::vector<Point> heroPoints = { heroPoint1, heroPoint2, heroPoint3, heroPoint4, heroPoint5 };

    if (!tauren->getIsDead() && hero->getIsAttacking()) {
        // Rect taurenRect = Rect(tauren->getPositionX(), tauren->getPositionY() - 40, 20, 40);
        Rect taurenRect = Rect(tauren->getPositionX() - 20, tauren->getPositionY() - 40, 20, 40);
        ...
    }
}

```

```

}

...

}

```

4.7 碰撞检测不严谨

CollisionDetection 类用关键点法做碰撞检测。事实上，这样的碰撞检测不严谨，如 Hero 和 Tauren 背靠背时，Tauren 攻击时斧头的关键点可能也落在 Hero 的矩形内，进而误判为攻击。

在 CollisionDetection 类的 update() 函数中判定碰撞前加入 Hero 与 Tauren 的朝向判定即可。朝向可用 ScaleX 判断，具体地：

(1) Hero: ScaleX = 1 朝右, ScaleX = -1 朝左。

(2) Tauren: ScaleX = 1 朝左, ScaleX = -1 朝右。

先在 CollisionDetection 类中实现判断 Hero 能否攻击到 Tauren 的函数 canHeroAttackTauren() 和判断 Tauren 能否攻击到 Hero 的函数 canTaurenAttackHero()。

```

// 判断 Hero 能否攻击到 Tauren
bool CollisionDetection::canHeroAttackTauren() {
    return (!doubleCompare(hero->getScaleX(), 1) && doubleCompare(hero->getPositionX(),
tauren->getPositionX()) <= 0)
        || (!doubleCompare(hero->getScaleX(), -1) && doubleCompare(hero->getPositionX(),
tauren->getPositionX()) >= 0);
}

// 判断 Tauren 能否攻击到 Hero
bool CollisionDetection::canTaurenAttackHero() {
    return (!doubleCompare(tauren->getScaleX(), 1) &&
doubleCompare(tauren->getPositionX(), hero->getPositionX()) >= 0)
        || (!doubleCompare(tauren->getScaleX(), -1) &&
doubleCompare(tauren->getPositionX(), hero->getPositionX()) <= 0);
}

```

```

void CollisionDetection::update(float dt) {
    if (canHeroAttackTauren()) {
        // Hero 攻击 Tauren
        ...
    }

    if (canTaurenAttackHero()) {
        // Tauren 攻击 Hero
        ...
    }
}

```

修改后，不再出现 Hero 与 Tauren 背对背攻击命中的情况。

4.8 连续点击技能

连续点击攻击按钮时，Hero 会持续回到 attack 动作的第 0 帧，导致 Hero 在原地抽搐，无法进行正确的攻击。

该 Bug 有如下两种修复方式：

(1) 用一个 bool 变量记录 Hero 是否正在播放 attack 动作，并在 attack 动作结束后复位。

缺点：attack 动作被打断时，可能无法及时复位 bool 变量。

(2) 为技能设置 CD，点击后技能按钮消失，直至 CD 结束才出现。

6. 游戏优化中会引入更多的技能，有些技能需要 CD，故此处采用方式 (2) 修复。

先在 Config.h 中指定攻击按钮、防御按钮的 tag 和攻击技能、防御技能的 CD。

```
// 技能按钮
static const int ATTACK_BUTTON_TAG = 201;
...
static const int DEFENSE_BUTTON_TAG = 202;
...

// 技能 CD
static const int ATTACK_CD = 60;
static const int DEFENSE_CD = 50;
```

在 Level1Scene 类中添加攻击技能、防御技能的 CD 属性。

```
// 技能 CD
int attackCD;
int defenseCD;
```

声明并实现更新所有技能按钮的函数 updateSkillButtons()。

```
// 更新所有技能按钮
void Level1Scene::updateSkillButtons() {
    if (attackCD) {
        attackCD++;
        if (attackCD >= ATTACK_CD) {
            attackCD = 0;
            this->getChildByTag(ATTACK_BUTTON_TAG)->setVisible(true);
        }
    }

    if (defenseCD) {
        defenseCD++;
        if (defenseCD >= DEFENSE_CD) {
            defenseCD = 0;
            this->getChildByTag(DEFENSE_BUTTON_TAG)->setVisible(true);
        }
    }
}
```

修改攻击按钮、防御按钮的回调函数，进入后启用相应的 CD，并将按钮设为不可见。

```
// 攻击按钮的回调函数
```



```

void Level1Scene::attackButtonCallback(Ref* pSender) {
    hero->switchState(State::ATTACK);

    // 启动攻击按钮 CD
    attackCD = 1;
    this->getChildByTag(ATTACK_BUTTON_TAG)->setVisible(false);
}

// 防御按钮的回调函数
void Level1Scene::defenseButtonCallback(Ref* pSender) {
    hero->switchState(State::DEFENSE);

    // 启动防御按钮 CD
    defenseCD = 1;
    this->getChildByTag(DEFENSE_BUTTON_TAG)->setVisible(false);
}

```

修改后，玩家点击攻击按钮后按钮消失，等待攻击技能 CD 结束后按钮再次出现，此时 Hero 不再会原地抽搐。

4.9 smitten 动画未执行

Hero 受伤后未播放 smitten 动画。

检查发现，Hero 类的 switchState()函数判断状态为 SMITTEN 时仅设置了 bool 变量，未播放 smitten 动画。补充播放 smitten 动画即可。

```

// 切换状态
void Hero::switchState(State _state) {
    if (_state == SMITTEN) {
        armature->getAnimation()->play("smitten");
        isHurt = true;
        isDefensing = false;
    }

    state = _state;
}

```

修改后，Hero 受伤会播放 smitten 动作。

4.10 attack 动画会被 smitten 动画打断

Hero 和 Tauren 的 attack 动画会被 smitten 动画打断，玩家可通过一直攻击持续打断 Tauren 的其它动作。

在 4.8 连续点击技能中取合适的攻击技能 CD，本次实验取 ATTACK_CD = 60。

修改后，玩家无法一直攻击，进而无法通过一直攻击持续打断 Tauren 的其它动作。

4.11 云朵移动动画衔接不上

云朵移动动画衔接不上，回到初始状态前有明显的停顿和画面差异。

检查发现，Level1Scene 类中采用两个云图层滚动的方式实现云朵移动动画。在

updateCloudBackground()函数中,对 cloudBackground1 应回到原点的时刻判断有误。事实上,无需推导 cloudBackground1 与屏幕的相对关系,只需在 cloudBackground2 到达 cloudBackground1 的初始位置时将 cloudBackground1 回到原点即可。

```
// 更新云背景
void Level1Scene::updateCloudBackground() {
    auto cloudBackground1 = (Sprite*)this->getChildByTag(CLOUD_BACKGROUND_TAG);
    auto cloudBackground2 = (Sprite*)this->getChildByTag(CLOUD_BACKGROUND_TAG +
1);
    //      if      (cloudBackground1->getPositionX()      >      -(3      /      2      *
cloudBackground1->getContentSize().width - visibleSize.width)) {
        if (doubleCompare(cloudBackground2->getPositionX(), visibleSize.width / 2) >= 0) {
            cloudBackground1->setPositionX(cloudBackground1->getPositionX() - 1);
        }
        else {
            cloudBackground1->setPositionX(visibleSize.width / 2);
        }
        cloudBackground2->setPositionX(cloudBackground1->getPositionX()
cloudBackground1->getContentSize().width);
    }
}
```

修改后,云朵移动动画衔接流畅,且可循环播放。

4.12 角色朝向切换

Hero 类的 update()函数和 Tauren 类的 update()函数中用

```
if (armature->getScaleX() != 1)
```

判断 ScaleX 的值是否为 1,用

```
armature->setScaleX(-1);
```

设置 ScaleX 的值。因计算机存储浮点数可能有误差,可能会导致 ScaleX 是一个非常接近 1 但与 1 不相等的数,进而条件不成立,影响角色朝向切换。

将浮点数的判等改为 doubleCompare()函数。

```
void Hero::update(float dt) {
    ...

    switch (state) {
        ...
        case MOVELEFT: {
            // if (!isDead && this->getPositionX() > 0 && !isHurt && !isAttacking) {
            if (!isDead && this->getPositionX() > 0 && !isAttacking) {
                if (!isRunning) {
                    armature->getAnimation()->play("run");
                    isRunning = true;
                }

                // if (armature->getScaleX() != 1) {
                if (doubleCompare(armature->getScaleX(), -1)) {
```

```

        armature->setScaleX(-1);
    }

    // this->setPositionX(this->getPositionX() - SPEED_X);
}

break;
}
case MOVERIGHT: {
    // if (!isDead && this->getPositionX() <
    Director::getInstance()->getVisibleSize().width && !isHurt && !isAttacking) {
        if (!isDead && this->getPositionX() <
    Director::getInstance()->getVisibleSize().width && !isAttacking) {
            if (!isRunning) {
                armature->getAnimation()->play("run");
                isRunning = true;
            }

            // if (armature->getScaleX() != -1) {
            if (doubleCompare(armature->getScaleX(), 1)) {
                armature->setScaleX(1);
            }

            // this->setPositionX(this->getPositionX() + SPEED_X);
        }

        break;
    }
    ...
}

```

修改后，角色的朝向切换不再出问题。

4.13 Hero 诈尸

若 Hero 或 Tauren 死后继续受伤，会播放 smitten 动作，进而“诈尸”。

在 Level1Scene 中，游戏胜负后，将游戏停下。

```

// 游戏胜利
void Level1Scene::workVictory() {
    pause = true;

    ...
}

// 游戏失败
void Level1Scene::workDefeat() {

```

```

    pause = true;

    hero->stopAllActions();
    taurenAI->stopAllActions();

    updateTaurenScoreboard(taurenScore + 1);
}

```

当且仅当 `pause = true` 时才执行 `update()` 函数的功能。

```

void Level1Scene::update(float dt) {
    if (pause) {
        return;
    }

    ...
}

```

修改后，Hero 或 Tauren 死后游戏暂停，即使再受伤也不会被处理，进而不会“诈尸”。

4.14 攻击、受伤后无法恢复 loading 动作

虽 Hero 类中设置了帧动画结束时的行为，即

```

// 帧动画
void Hero::onFrameEvent(cocostudio::Bone* bone, const std::string& evt, int originFrameIndex,
int currentFrameIndex) {
    if (evt == "attack_end") {
        armature->getAnimation()->play("loading");
        isAttacking = false;
    }
    else if (evt == "smitten_end") {
        armature->getAnimation()->play("loading");
        isHurt = false;
    }
    else if (evt == "defense_end") {
        armature->getAnimation()->play("loading");
        isDefensing = false;
    }
}
}

```

但 Hero 攻击、受伤后并未播放 loading 动画，而是直接卡在原地。

检查发现 Hero 的骨骼动画中，attack 动作和 smitten 动作的最后一帧未添加对应的帧事件。

在 Cocos Studio 的 Animation Editor 中为 attack 动作和 smitten 动作的最后一帧分别添加帧事件 `attack_end` 和 `smitten_end` 即可。以 attack 动作为例，如图 4.14.1 所示。

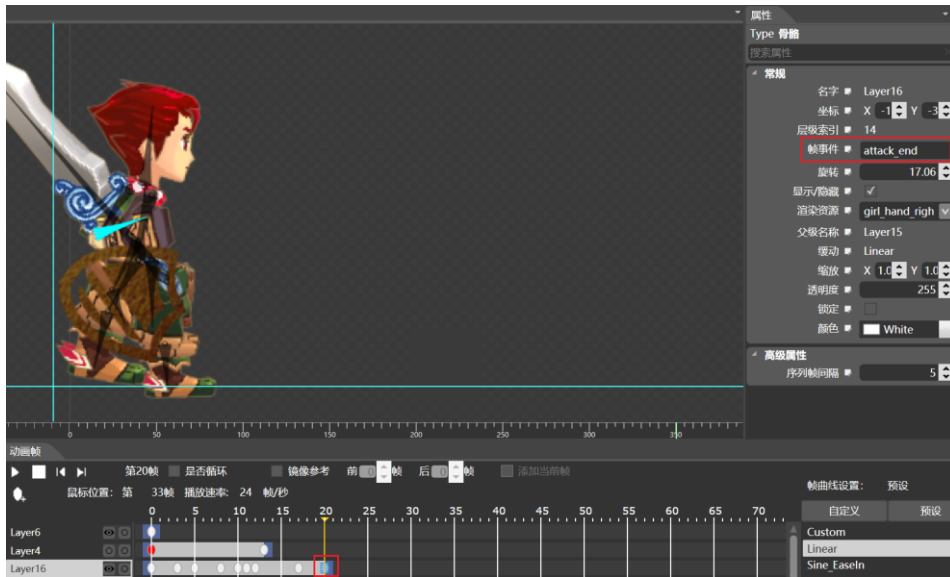


图 4.14.1: 为 attack 动作的最后一帧添加帧事件 attack_end

4.15 Hero 很靠近 Tauren 时持续受伤

Hero 很靠近 Tauren 时，可能会因 Hero 的矩形区域始终包含 Tauren 的武器的关键点而持续受伤。

Hero 每次受伤后上锁，上锁期间不再受伤，过一段时间解锁即可。

先在 Config.h 中指定 Hero 相邻两次受伤的最小时间间隔，即上锁时间。

```
static const int HERO_SMITTEN_LOCK = 50; // Hero 相邻两次受伤的最小时间间隔
```

为 Hero 类添加受伤的锁计时器 smittenLock，并在 init()函数中初始化。

```
bool Hero::init(int _maxHP, Point position) {
    ...

    smittenLock = 0;

    ...
}
```

在 update()函数中更新 smittenLock。

```
void Hero::update(float dt) {
    ...

    // 受伤的锁
    if (smittenLock) {
        smittenLock++;
        if (smittenLock >= HERO_SMITTEN_LOCK) {
            smittenLock = 0;
        }
    }
}
```

在 playHurt()函数中增加对 smittenLock 的判定。

```
// 受伤
```

```

void Hero::playHurt() {
    // 上受伤锁期间不受伤
    if (smittenLock) {
        return;
    }

    smittenLock = 1;

    ...
}

```

修改后，Hero 很靠近 Tauren 时也不会持续受伤。

4.16 角色受伤时攻击被打断

Hero 或 Tauren 受伤时会打断 attack 动作，导致无法触发 attack_end 的帧事件。

检查发现，Hero 类和 Tauren 类的 update()函数的 switch()中，ATTACK 分支的处理的判定多了!isHurt，去掉即可。以 Hero 类为例。

```

void Hero::update(float dt) {
    ...

    switch (state) {
        ...
        case ATTACK: {
            // if (!isDead && !isHurt) {
            if (!isDead) {
                armature->getAnimation()->play("attack");
                isAttacking = true;

                // 播放攻击音效
                SimpleAudioEngine::getInstance()->playEffect(HIT_METAL_PATH.c_str());
            }

            break;
        }
        ...
    }

    ...
}

```

5. 游戏音乐

5.1 加载与播放音乐的函数

5.1.1 预加载音乐的函数

BasicScene 类声明了预加载音效的 preloadSoundEffect() 函数、预加载背景音乐的 preloadBackgroundMusic() 函数和预加载场景音乐的虚函数 preloadSounds()。

```
// 预加载音效
void preloadSoundEffect(std::string soundEffectPath);

// 预加载背景音乐
void preloadBackgroundMusic(std::string backgroundMusicPath);

// 预加载音乐
virtual void preloadSounds();
```

其中

```
// 预加载音效
void BasicScene::preloadSoundEffect(std::string soundEffectPath) {
    SimpleAudioEngine::getInstance()->preloadEffect(soundEffectPath.c_str());
}

// 预加载背景音乐
void BasicScene::preloadBackgroundMusic(std::string backgroundMusicPath) {

SimpleAudioEngine::getInstance()->preloadBackgroundMusic(backgroundMusicPath.c_str());
}
```

5.1.2 播放音乐的函数

BasicScene 类实现了播放、暂停、恢复、停止音效或背景音乐的函数。

```
// 播放音效
void BasicScene::playSoundEffect(std::string soundEffectPath) {
    SimpleAudioEngine::getInstance()->playEffect(soundEffectPath.c_str());
}

// 播放背景音乐
void BasicScene::playBackgroundMusic(std::string backgroundMusicPath) {
    SimpleAudioEngine::getInstance()->playBackgroundMusic(backgroundMusicPath.c_str());
}

// 暂停音效
void BasicScene::pauseSoundEffect() {
    SimpleAudioEngine::getInstance()->pauseAllEffects();
}

// 暂停背景音乐
void BasicScene::pauseBackgroundMusic() {
```



```

        SimpleAudioEngine::getInstance()->pauseBackgroundMusic();
    }

    // 恢复音效
void BasicScene::resumeSoundEffect() {
    SimpleAudioEngine::getInstance()->resumeAllEffects();
}

    // 恢复背景音乐
void BasicScene::resumeBackgroundMusic() {
    SimpleAudioEngine::getInstance()->resumeBackgroundMusic();
}

    // 停止音效
void BasicScene::stopSoundEffect() {
    SimpleAudioEngine::getInstance()->stopAllEffects();
}

    // 停止背景音乐
void BasicScene::stopBackgroundMusic() {
    SimpleAudioEngine::getInstance()->stopBackgroundMusic();
}

```

5.2 关卡音乐的选择

5.2.1 音效选择

Hero 的武器是剑，攻击的音效选取敲击金属声。

Tauren 的武器是斧头，斧头前有蓝色水晶，如图 5.2.1.1 所示。不妨认为它是冰属性的，攻击的音效取冰碎的声音（本次实验用玻璃碎的声音代替）。



图 5.2.1.1: Tauren 的斧头

因角色准备攻击到挥动武器有一定延迟，可在音效前加一段空白以填充。

5.2.2 背景音乐选择

《侠客行》设 3 关，技能的丰富程度、难度递增。

因优化后游戏的美工偏远古、神秘、埃及，且《侠客行》是格斗类游戏，应选择较燃的音乐作为 bgm。本次实验选择《妖精的尾巴》的 3 首 OP 分别作为 3 关的 bgm。

（1）第一关：高梨康治 - 魔法発動

- (2) 第二关：高梨康治 - Dokuryuu No Cobra
- (3) 第三关：高梨康治 - FAIRY TAIL メインテーマ

5.3 加载关卡 1 的音乐

5.3.1 预加载音乐

在 Config.h 中指明 bgm 和音效的路径。

```
// bgm
static const std::string MOFA_PATH = "sounds/mofa.mp3";
static const std::string DOKU_PATH = "sounds/doku.mp3";
static const std::string FAIRY_PATH = "sounds/fairy.mp3";

// 音效
static const std::string HIT_METAL_PATH = "sounds/hitMetal.mp3";
static const std::string GLASS_BREAK_PATH = "sounds/glassBreak.mp3";
```

Level1Scene 类重写 BasicScene 类的 preloadSounds()函数，预加载本关卡用到的音乐。

```
// 预加载音乐
void Level1Scene::preloadSounds() {
    preloadBackgroundMusic(MOFA_PATH);
    preloadSoundEffect(HIT_METAL_PATH);
    preloadSoundEffect(GLASS_BREAK_PATH);
}
```

5.3.2 加载背景音乐

在 Level1Scene 的 init()函数中预加载音乐并播放 bgm。因初始时场景暂停，故先暂停 bgm。

```
bool Level1Scene::init() {
    ...

    pause = true;

    ...

    preloadSounds();

    playBackgroundMusic(MOFA_PATH);

    pauseBackgroundMusic();
}
```

在键盘监听的回调函数中，设置空格键的行为，开始游戏。

```
// 键盘监听的回调函数
void Level1Scene::onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event) {
    switch (keyCode) {
        ...
        case EventKeyboard::KeyCode::KEY_SPACE: { // 空格:
            pause = false;
        }
```

```

        // 更新 bgm 状态等
        resumeBackgroundMusic();
        taurenAI->startAction();
        break;
    }
    case EventKeyboard::KeyCode::KEY_P: { // P: 暂停
        pause ^= 1;

        // 更新 bgm 状态等
        if (pause) {
            pauseBackgroundMusic();
            taurenAI->stopAction();
        }
        else {
            resumeBackgroundMusic();
            taurenAI->startAction();
        }
        break;
    }
    ...
}
}

```

其中 TaurenAI 类的 startAction()函数和 stopAction()函数的实现如下。

```

// 启动 TaurenAI
void TaurenAI::startAction() {
    auto sequence = Sequence::create(
        DelayTime::create(0.8f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveLeft, this)),
        DelayTime::create(1.0f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),
        DelayTime::create(0.3f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveRight, this)),
        // DelayTime::create(0.7f),
        DelayTime::create(0.85f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::stand, this)),
        DelayTime::create(0.5f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),
        nullptr
    );
    RepeatForever* action = RepeatForever::create(sequence);
    this->runAction(action);
}

```

```
// 停止 TaurenAI
void TaurenAI::stopAction() {
    this->stopAllActions();
}
```

5.3.3 加载音效

在 Hero 类的 update()函数的 switch 中，为 ATTACK 状态添加播放攻击音效。

```
void Hero::update(float dt) {
    ...

    switch (state) {
        ...
        case ATTACK: {
            if (!isDead && !isHurt) {
                armature->getAnimation()->play("attack");
                isAttacking = true;

                // 播放攻击音效
                SimpleAudioEngine::getInstance()->playEffect(HIT_METAL_PATH.c_str());
            }

            break;
        }
        ...
    }
}
```

同理为 Tauren 类的 update()函数添加。

6. 游戏优化

6.1 游戏策划

6.1.1 剧情

(1) 游戏剧情:

几百年前,人族和牛头人族原本在这片土地上和平共生。但随着能源的消耗,人族与牛头人族的冲突愈发激烈。今天,人族与牛头人族正在争夺淡水资源,一场大战一触即发。

你将代表人族,与牛头人族派出的代表对战,胜者将获得淡水资源。

你能为人族夺得淡水资源吗?

(2) 通关剧情:

人族夺得了淡水资源。感谢你为人族的延续做出的贡献。

6.1.2 Tauren AI 优化

Tauren 倾向于朝玩家所在的方向运动。

6.1.3 随机性优化

为增强游戏随机性,有如下优化:

(1) 攻击有 20%的概率二连击,有 5%的概率三连击。

(2) 攻击有 20%的概率暴击。

(3) 攻击有 20%的概率 Miss。

上述概率相互独立。

6.1.4 技能设置

Hero 技能:

(1) 平 A: 普通近战攻击。

(2) 防御: 举盾期间抵消部分伤害。

(3) 击退: 远程攻击,可击退敌人。

(4) 【大招】弹幕: Hero 所在位置上方出现弹幕发射器,往四周发射弹幕,弹幕会对敌人造成伤害。

Tauren 技能:

(1) 平 A: 普通近战攻击。

(2) 护甲: 出现一个可以抵挡一次攻击的护甲。

(3) 隐身: 一段时间内无法被敌人看见。

(4) 【大招】击飞: 全屏击飞,敌人离 Tauren 越近伤害越高,敌人落地掉血。

6.1.5 关卡设置

《侠客行》设 3 个关卡,丰富度、难度递增,主要体现在技能上。具体地:

关卡	Hero				Tauren			
	平 A	防御	击退	弹幕	平 A	护甲	隐身	击飞
一	√	√			√	√		
二	√	√	√		√	√	√	
三	√	√	√	√	√	√	√	√

此外,3 个关卡的 Tauren AI 会越来越“聪明”,表现为根据 Hero 与 Tauren 的相对位置做决策的时间缩短。具体地:

(1) 第一关: 决策时间 400。

(2) 第二关: 决策时间 300。

(3) 第三关: 决策时间 200。

6.2 Tauren AI 优化

原 Tauren AI 是一个重复播放的动作序列，指定参数后会朝一个方向有明显的倾向性，无法根据 Tauren 与 Hero 的相对位置采取策略。

取朝向比例 1 : 0.81 的两个动作序列，每隔一段时间根据 Tauren 与 Hero 的相对位置决定应采用哪个动作序列。注意在 Tauren 状态为 STAND 或 SMITTEN 时切换动作序列，以免打断其它动作。

先在 Config.h 中指定 Tauren 切换动作序列的时间间隔 TAUREN_SWITCH_ACTION。它反映 Tauren AI 的灵敏度，每关的参数可不同，故不设为 const。

```
static int TAUREN_SWITCH_ACTION = 300; // Tauren 切换动作序列的时间间隔
```

在 TaurenAI.h 中定义 Tauren 朝向的枚举类型 TaurenDirection。

```
enum TaurenDirection {  
    FACELEFT,  
    FACERIGHT  
};
```

为 TaurenAI 类添加 Tauren 倾向的方向 taurenDirection 和 Tauren 改变倾向计时器 taurenSwitchAction。

```
TaurenDirection taurenDirection; // Tauren 倾向的方向  
int taurenSwitchAction; // Tauren 改变倾向计时器
```

在 init()函数中初始化。

```
bool TaurenAI::init(Node* parent) {  
    ...  
  
    taurenDirection = TaurenDirection::FACELEFT;  
    taurenSwitchAction = 0;  
  
    ...  
}
```

修改 startAction()函数，为其添加默认参数，并实现根据 Tauren 的倾向选择动作序列。

```
// 启动 TaurenAI  
void startAction(TaurenDirection _taurenDirection = TaurenDirection::FACELEFT);  
  
// 启动 TaurenAI  
void TaurenAI::startAction(TaurenDirection _taurenDirection) {  
    taurenDirection = _taurenDirection;  
  
    Sequence* sequence;  
    if (taurenDirection == TaurenDirection::FACELEFT) {  
        sequence = Sequence::create(  
            DelayTime::create(0.8f),  
            CallFunc::create(CC_CALLBACK_0(TaurenAI::moveLeft, this)),  
            DelayTime::create(1.0f),  
            CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),  
            DelayTime::create(0.3f),  
            CallFunc::create(CC_CALLBACK_0(TaurenAI::moveRight, this)),  
            ...  
        );  
    }  
}
```

```

        DelayTime::create(0.81f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::stand, this)),
        DelayTime::create(0.5f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),
        nullptr
    );
    RepeatForever* action = RepeatForever::create(sequence);
    this->runAction(action);
}
else {
    sequence = Sequence::create(
        DelayTime::create(0.8f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveRight, this)),
        DelayTime::create(1.0f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),
        DelayTime::create(0.3f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::moveLeft, this)),
        DelayTime::create(0.81f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::stand, this)),
        DelayTime::create(0.5f),
        CallFunc::create(CC_CALLBACK_0(TaurenAI::attack, this)),
        nullptr
    );
    RepeatForever* action = RepeatForever::create(sequence);
    this->runAction(action);
}
}
}

```

在 setAI()函数中不再调用 startAction()。

```

void TaurenAI::setAI(Hero* _hero, Tauren* _tauren) {
    hero = _hero;
    tauren = _tauren;

    this->scheduleUpdate();

    // startAction(TaurenDirection::FACELEFT);
}

```

update()函数完成 Tauren 倾向的更新。

```

void TaurenAI::update(float dt) {
    ...

    taurenSwitchAction++;
    if (taurenSwitchAction >= TAUREN_SWITCH_ACTION) {
        if (tauren->getState() == State::STAND || tauren->getState() == State::SMITTEN) {
            if (doubleCompare(hero->getPositionX(), tauren->getPositionX()) <= 0) {

```

```

        stopAction();
        startAction(TaurenDirection::FACELEFT);
    }
    else {
        stopAction();
        startAction(TaurenDirection::FACERIGHT);
    }

    taurenSwitchAction = 0;
}
}
}

```

此外还需在 Level1Scene 类的 init()函数中指定本关卡的 Tauren AI 的灵敏度。

```

bool Level1Scene::init() {
    ...

    // Tauren AI
    TAUREN_SWITCH_ACTION = 400;
    ...

    ...
}

```

并修改键盘监听的回调函数中空格键（开始游戏）和 P 键（暂停游戏）的实现。

```

// 键盘监听的回调函数
void Level1Scene::onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event) {
    switch (keyCode) {
        ...
        case EventKeyboard::KeyCode::KEY_SPACE: { // 空格:
            pause = false;

            // 更新 bgm 状态等
            resumeBackgroundMusic();
            taurenAI->startAction();
            break;
        }
        case EventKeyboard::KeyCode::KEY_P: { // P: 暂停
            pause ^= 1;

            // 更新 bgm 状态等
            if (pause) {
                pauseBackgroundMusic();
                taurenAI->stopAction();
            }
            else {

```



```

        resumeBackgroundMusic();
        taurenAI->startAction(taurenAI->getTaurenDirection());
    }
    break;
}
...
}
}

```

优化后，Tauren AI 每隔一段时间根据 Tauren 与 Hero 的相对位置决定 Tauren 的倾向方向，实现 Tauren 追着 Hero 跑的效果。

6.3 随机性优化

为增强游戏随机性，有如下优化：

- (1) 攻击有 20%的概率二连击，有 5%的概率三连击。
- (2) 攻击有 20%的概率暴击。非暴击的扣血为橙色，暴击的扣血为红色。
- (3) 攻击有 20%的概率 Miss。

上述概率相互独立。

实现上，应先实现 Miss，后实现连击，再实现暴击。

修改 Hero 类和 Tauren 类的 displayBloodTip()函数。以前者为例。

```

// 显示扣血
void Hero::displayBloodTip(int basicDamage) {
    // 20% Miss
    int missRand = generateRandomInteger(1, 10);
    if (missRand <= 2) {
        auto label = Label::createWithBMFont(TIPS_FNT_PATH, "Miss");
        label->setColor(Color3B::GREEN);
        label->setPosition(Point(20 + generateRandomInteger(0, 80), 10 +
generateRandomInteger(0, 80)));
        label->runAction(
            Sequence::create(
                MoveBy::create(0.7f, Point(0, 30)),
                CallFunc::create(CC_CALLBACK_0(Hero::bloodTipFlyEnd, this, label)),
                nullptr
            )
        );

        this->addChild(label, 15);

        return;
    }

    // 20% 二连击, 5% 三连击
    int hitCount = 1; // 连击次数
    int hitRand = generateRandomInteger(1, 100);

```

```

        if (hitRand <= 5) {
            hitCount = 3;
        }
        else if (hitRand <= 20) {
            hitCount = 2;
        }

        // hitCount = 1; // 取消连击

        for (int i = 0; i < hitCount; i++) {
            int damage;

            // 20% 暴击
            int criticalRand = generateRandomInteger(1, 10);
            if (criticalRand <= 2) {
                damage = basicDamage + generateRandomInteger(10, 20);
            }
            else {
                damage = basicDamage + generateRandomInteger(0, 10);
            }
            // damage = basicDamage; // 取消随机攻击力
            setHP(HP - damage);

            auto label = Label::createWithBMFont(TIPS_FNT_PATH, StringUtils::format("-%d",
damage));
            if (criticalRand <= 2) {
                label->setColor(Color3B::RED); // 暴击为红色
            }
            else {
                label->setColor(Color3B::ORANGE); // 非暴击为橙色
            }
            label->setPosition(Point(20 + generateRandomInteger(0, 80), 10 +
generateRandomInteger(0, 80)));
            label->runAction(
                Sequence::create(
                    MoveBy::create(0.7f, Point(0, 30)),
                    CallFunc::create(CC_CALLBACK_0(Hero::bloodTipFlyEnd, this, label)),
                    nullptr
                )
            );

            this->addChild(label, 15);
        }
    }
}

```

优化后，运行结果如图 6.3.1、图 6.3.2 和图 6.3.3 所示。



图 6.3.1: Miss



图 6.3.2: 三连击

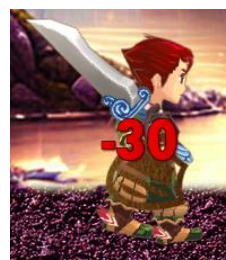


图 6.3.3: 暴击

6.4 开始界面

6.4.1 剧情绘制

因 Cocos-2dx 的 Label 过于鸡肋，故采用图片的方式实现剧情。

先绘制背景剧情和通关剧情的图片。如图 6.4.1.1 和图 6.4.1.2 所示。



图 6.4.1.1: 背景剧情

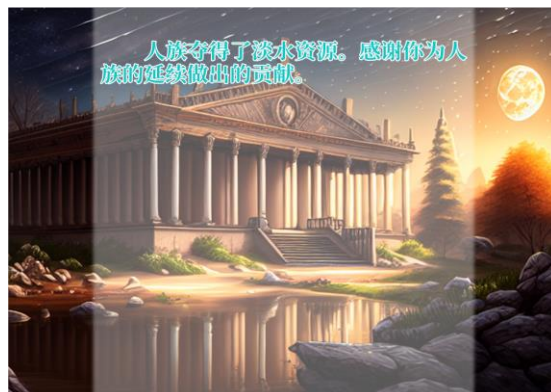


图 6.4.1.2: 通关剧情

去掉背景，仅保留文字部分，分别保存为 StartStory.png 和 EndStory.png。如图 6.4.1.3 所示。



StartStory.png



EndStory.png

图 6.4.1.3: 保存文字部分图片

6.4.2 开始界面类

计划实现如图 6.4.2.1 所示的，剧情从上往下依次出现的效果。

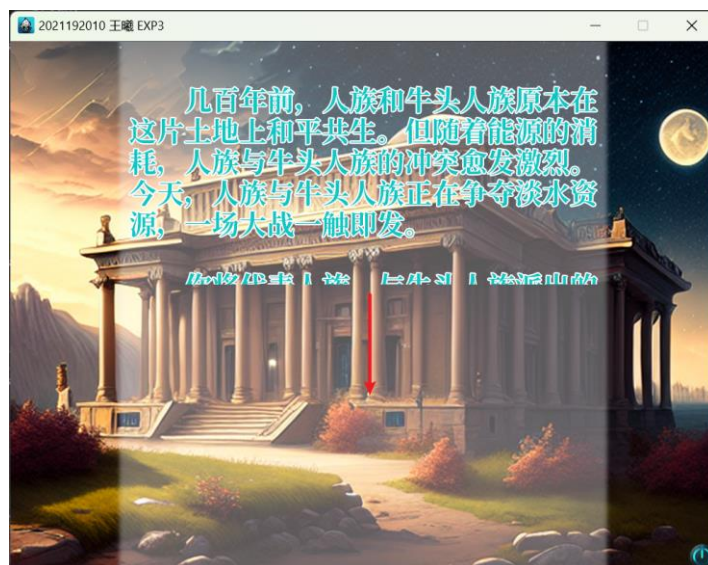


图 6.4.2.1: 剧情从上往下出现

先在 Config.h 中指定剧情/提示的出现速度（显示的百分比的增量）HINT_DELTA_PERCENT、开始剧情的路径 START_STORY_PATH 和通关剧情的路径 END_STORY_PATH。

```
// 剧情/提示
static const float HINT_DELTA_PERCENT = 0.01; // 剧情/提示出现速度
static const std::string START_STORY_PATH = "hints/StartStory.png";
static const std::string END_STORY_PATH = "hints/EndStory.png";
```

为开始界面类 StartScene 添加成员变量：剧情的标签 story 和剧情显示百分比 storyPercent。

```
Sprite* story;

float storyPercent; // 剧情显示百分比
```

在 init()函数中加载背景和剧情，将剧情的 y 值设在可视区域外，即初始时不可见。一打开游戏即开始显示剧情会降低玩家的体验感，故初始时游戏暂停。

```
bool StartScene::init() {
    ...

    pause = true;
    storyPercent = 0;

    ...

    // ===== 加载场景 =====
    loadImage(START_BACKGROUND_PATH, Point(visibleSize.width / 2, visibleSize.height /
2), 0);

    loadImage(START_STORY_PATH, Point(visibleSize.width / 2, 2 * visibleSize.height), 0, 1);
    // y 在可视区域外
    story = (Sprite*)getChildByTag(1);
```

```
...
}
```

update()函数中用类似于血条类的方式更新剧情图片的显示区域，剧情完全显示后，暂停游戏。

```
void StartScene::update(float dt) {
    if (pause) {
        return;
    }

    if (doubleCompare(storyPercent, 1) >= 0) {
        pause = true;

        return;
    }

    storyPercent += HINT_DELTA_PERCENT;
    story->setTextureRect(Rect(0, 0, visibleSize.width, visibleSize.height * storyPercent));
    story->setPositionY(visibleSize.height / 2 + (1 - storyPercent) * visibleSize.height / 2);
}
```

刚进入游戏的玩家不知道应如何操作，故加位提示
先在 Config.h 中指定提示信息的内容和 tag。

```
// 剧情/提示
static const std::string HINT_BACKGROUND_PATH = "hints/HintBackground.png";
static const int START_AND_PAUSE_HINT_TAG = 301;
static const std::string START_AND_PAUSE_HINT = "Press SPACE to start or P to pause/resume.";
static const std::string REPLAY_AND_NEXT_HINT = "Press R to replay or N to the next level.";
```

在 BasicScene 类声明并实现加载键位提示的函数 loadKeyboardHint()，用于将键位提示显示在画面中间偏下方。类似于 Level1Scene 类中攻击按钮，为提示信息添加闪烁效果，吸引玩家注意。

```
// 加载键位提示
void BasicScene::loadKeyboardHint(std::string hintWords, int z, int tag) {
    auto keyboardHint = Label::createWithBMFont(TIPS_FNT_PATH, hintWords);
    if (keyboardHint) {
        keyboardHint->setPosition(Point(visibleSize.width / 2, 2 *
keyboardHint->getContentSize().height));
        keyboardHint->setScale(0.7);
        keyboardHint->setColor(Color3B::MAGENTA);
        if (~tag) {
            keyboardHint->setTag(tag);
        }

        auto actionSequence = Sequence::create(
```

```

        FadeOut::create(1),
        FadeIn::create(1),
        nullptr
    );
    auto action = RepeatForever::create(actionSequence);
    keyboardHint->runAction(action);

    this->addChild(keyboardHint, z);
}
else {
    CCLOG(("Font \"\" + TIPS_FNT_PATH + "\" loaded failed.").c_str());
}
}

```

在 StartScene 类的 init()函数中加载“开始游戏或暂停/恢复”的键位提示。

```

bool StartScene::init() {
    ...

    loadKeyboardHint(START_AND_PAUSE_HINT, 10, START_AND_PAUSE_HINT_TAG);

    ...
}

```

剧情完全显示后，加载“重玩或下一关”的键位提示。

```

void StartScene::update(float dt) {
    ...

    if (doubleCompare(storyPercent, 1) >= 0) {
        pause = true;

        loadKeyboardHint(REPLAY_AND_NEXT_HINT, 10);

        return;
    }

    ...
}

```

6.4.3 场景切换

StartScene 类重写 onKeyPressed()函数，实现如下键盘交互：

- (1) 空格：开始显示剧情，并移除“开始游戏或暂停/恢复”的键位提示。
- (2) P：暂停剧情。
- (3) R：剧情显示完全后，重玩。
- (4) N：剧情显示完全后，下一关。

后续关卡的场景切换的键盘交互类似，后续不再赘述。

```

// 键盘监听的回调函数
void StartScene::onKeyPressed(EventKeyboard::KeyCode keyCode, Event* event) {

```

```

switch (keyCode) {
    case EventKeyboard::KeyCode::KEY_ESCAPE: { // ESC: 关闭
        Director::getInstance()->end();
        break;
    }
    case EventKeyboard::KeyCode::KEY_SPACE: { // 空格:
        pause = false;

        auto startAndPauseHint = getChildByTag(START_AND_PAUSE_HINT_TAG);
        if (startAndPauseHint) {
            removeChildByTag(START_AND_PAUSE_HINT_TAG);
        }

        // 更新 bgm 状态等
        break;
    }
    case EventKeyboard::KeyCode::KEY_P: { // P: 暂停
        pause ^= 1;

        // 更新 bgm 状态等
        break;
    }
    case EventKeyboard::KeyCode::KEY_N: { // N: 下一关
        auto newScene = Level1Scene::createScene();
        Director::getInstance()->pushScene(newScene);

        break;
    }
    case EventKeyboard::KeyCode::KEY_R: { // R: 重玩
        auto newScene = StartScene::createScene();
        Director::getInstance()->replaceScene(newScene);

        break;
    }
}
}

```

优化后，玩家进入游戏后，先显示如图 6.4.3.1 所示的开始界面。玩家按空格键开始加载剧情，如图 6.4.3.2 所示。剧情加载完全，如图 6.4.3.3 所示。玩家按 N 键进入第一关，如图 6.4.3.4 所示。

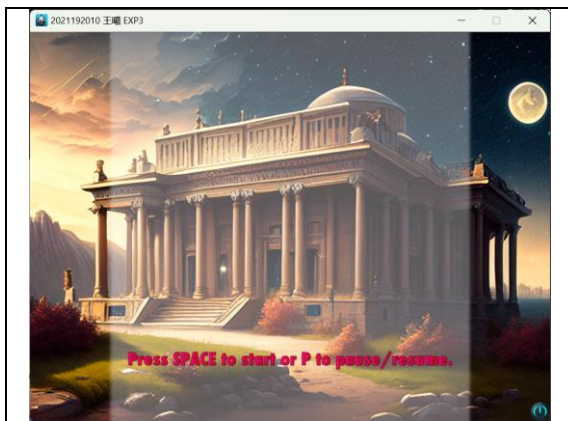


图 6.4.3.1: 开始界面



图 6.4.3.2: 剧情加载中

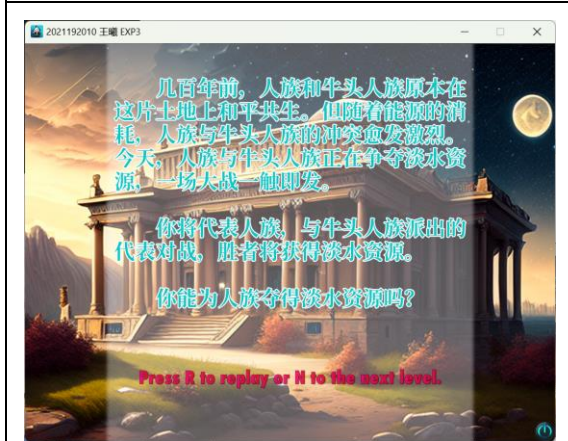


图 6.4.3.3: 剧情加载完全

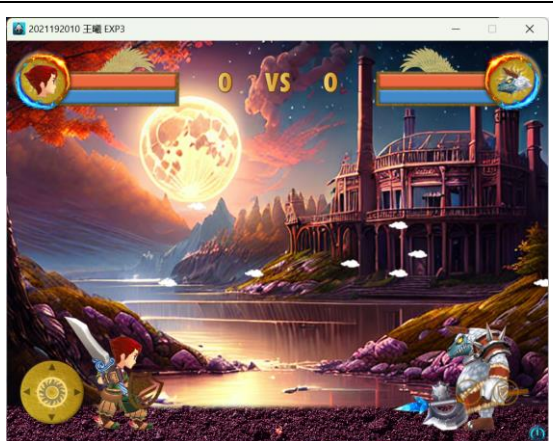


图 6.4.3.4: 进入第一关

6.5 独立计分板

计分板应记录 Hero 和 Tauren 在所有关卡中的得分，故将计分板的数值独立到 Config.h 中。

```
// 计分板
```

```
static int heroScore = 0;
```

```
static int taurenScore = 0;
```

在 Level1Scene 类的 init() 函数中根据已有得分加载计分板即可。

```
bool Level1Scene::init() {
```

```
...
```

```
// 计分板
```

```
loadHeroScoreboard(heroScore, Point(visibleSize.width / 2 - 100, visibleSize.height - 80), 5);
```

```
loadTaurenScoreboard(taurenScore, Point(visibleSize.width / 2 + 100, visibleSize.height - 80), 5);
```

```
...
```

```
}
```

6.6 第一关

《侠客行》第一关的参数：

关卡	Hero				Tauren			
	平 A	防御	击退	弹幕	平 A	护甲	隐身	击飞
一	√	√			√	√		

Hero 血量：100。

Tauren 血量：100。

Tauren AI 的决策时间：400。

Hero 的技能、Tauren 的平 A 已实现，下面实现 Tauren 的护甲。

6.6.1 Tauren 技能 – 护甲

Tauren 的技能 - 护甲：

(1) CD：400。

(2) 效果：在 Tauren 身上出现一个蓝色的盾牌，可为 Tauren 抵挡一次攻击。Tauren 受到一次攻击后，盾牌消失。

为 Tauren 类添加成员变量：是否开启护甲 isDefensing 并在 init()函数中初始化。

在 playHurt()函数中添加护甲的判定。

```
// 受伤
void Tauren::playHurt() {
    // Tauren 护甲
    if (isDefensing) {
        isDefensing = false;
        return;
    }

    displayBloodTip(BASIC_DAMAGE);
    this->switchState(SMITTEN);
}
```

虽 Tauren 的盾牌跟随 Tauren 移动，但盾牌本身归属场景类。若将其作为 Tauren 类的子节点，则需处理局部坐标到世界坐标的转换，较繁琐。故本次实验在 Level1Scene 类中加载和更新 Tauren 的盾牌。具体地，先加载 Tauren 的盾牌，将其置于合适的高度，并隐藏。更新时，根据 Tauren 是否开启护甲技能决定盾牌是否显示。

```
// 加载 Tauren 的盾牌
void Level1Scene::loadTaurenShield() {
    taurenShield = Sprite::create(TAUREN_SHIELD_PATH);
    taurenShield->setPositionY(100);
    taurenShield->setVisible(false);
    this->addChild(taurenShield, 12);
}

// 更新 Tauren 的盾牌的位置
void Level1Scene::updateTaurenShieldPosition() {
    Point taurenPoint =
tauren->getArmature()->getBone("body")->getDisplayRenderNode()->convertToWorldSpaceAR(Point(0,
0));
    taurenShield->setPositionX(taurenPoint.x);
}
```

```
        if (tauren->getIsDefensing()) {
            taurenShield->setVisible(true);
        }
        else {
            taurenShield->setVisible(false);
        }
    }

void Level1Scene::update(float dt) {
    ...

    updateTaurenShieldPosition();
}
```

注意 Tauren 死亡，即游戏胜利时，需将 Tauren 的盾牌设为不可见。

```
// 游戏胜利
void Level1Scene::workVictory() {
    ...

    taurenShield->setVisible(false);

    ...
}
```

优化后，运行结果如图 6.6.1.1 所示。玩家需用一次攻击打破 Tauren 的盾牌，才可攻击到 Tauren。



图 6.6.1.1：开启护甲技能的 Tauren

6.7 第二关

《侠客行》第二关的参数：

关卡	Hero				Tauren			
	平 A	防御	击退	弹幕	平 A	护甲	隐身	击飞
二	√	√	√		√	√	√	

Hero 血量：100。

Tauren 血量：150。

Tauren AI 的决策时间：300。

6.7.1 子弹类

虽 Hero 的击退技能只涉及沿 x 轴飞行的子弹，但后续的弹幕技能涉及向各个方向飞行的子弹，故实现一个支持向各个方向飞行的子弹类。

为增加丰富性，子弹有如图 6.7.1.1 所示的 4 种贴图，每次生成时随机取一种贴图。

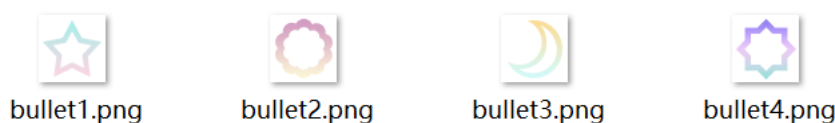


图 6.7.1.1: 子弹贴图

先在 Config.h 中指定子弹贴图的路径。

```
static const std::vector<std::string> BULLET_PATHS = {  
    "SkillItems/bullet1.png",  
    "SkillItems/bullet2.png",  
    "SkillItems/bullet3.png",  
    "SkillItems/bullet4.png"  
};
```

子弹类 Bullet 继承于 Layer 类，包含成员变量：子弹 bullet、子弹速度 speed、子弹基础伤害 damage、子弹是否有击退效果 knockback。

```
Sprite* bullet;  
Vec2 speed;  
int damage;  
bool knockback; // 有击退效果
```

Bullet 类声明并实现用于将速度正则化为给定模长的函数 normalizeSpeed() 和用于删除子弹的函数 removeBullet()。

```
void Bullet::normalizeSpeed(float _speed) {  
    assert(speed.getLength() > 0);  
    speed *= _speed / speed.getLength();  
}  
  
void Bullet::removeBullet() {  
    this->unscheduleUpdate();  
    this->removeChild(bullet);  
}
```

update() 函数中根据子弹的速度更新子弹位置。

```
void Bullet::update(float dt) {  
    if (bullet) {  
        bullet->setPosition(bullet->getPosition() + speed);  
    }  
}
```

6.7.2 Hero 技能 – 击退

Hero 的技能 – 击退：

(1) CD: 200。

(2) 效果：Hero 往面朝的方向发射一颗子弹，子弹击中 Tauren 后不仅会让它受伤，还会将其朝子弹飞行方向击退。若 Tauren 此时为防御状态，则子弹也可击破盾牌。

先在 Config.h 中指定击退技能的 CD 和子弹的参数。

```
// HERO 技能 CD  
...
```

```
static const int KNOCKBACK_CD = 200;
```

```
// 参数
```

```
...
```

```
static const int BULLET_BASIC_DAMAGE = 10; // 子弹的基础攻击力
```

```
static const int BULLET_SPEED = 10; // 子弹速度的模
```

```
static const float BULLET_FORCE = 5; // 子弹击退距离是其速度的 x 分量的倍数
```

修改 Tauren 类的 playHurt()函数，传入一个参数 bulletDamage 表示子弹的基础伤害，为 0 时表示非子弹造成的伤害。playHurt()函数根据伤害来源是否是子弹调用不同的扣血函数，其中子弹调用的扣血函数 displayBloodTip()无连击、无暴击，但在基础伤害的基础上有随机额外伤害。

```
// 受伤
```

```
void Tauren::playHurt(int bulletDamage) {
```

```
    // Tauren 护甲
```

```
    if (isDefensing) {
```

```
        isDefensing = false;
```

```
        return;
```

```
    }
```

```
    if (bulletDamage) { // 子弹
```

```
        displayBloodTip(bulletDamage);
```

```
    }
```

```
    else { // 平 A
```

```
        displayBloodTips(BASIC_DAMAGE);
```

```
    }
```

```
    this->switchState(SMITTEN);
```

```
}
```

```
// 显示单个扣血 (子弹)
```

```
void Tauren::displayBloodTip(int basicDamage) {
```

```
    int damage = basicDamage + generateRandomInteger(0, 5);
```

```
    setHP(HP - damage);
```

```
    auto label = Label::createWithBMFont(TIPS_FNT_PATH, StringUtils::format("-%d",  
damage));
```

```
    label->setColor(Color3B::ORANGE);
```

```
    label->setPosition(Point(20 + generateRandomInteger(0, 80), 10 +  
generateRandomInteger(0, 80)));
```

```
    label->runAction(  
        Sequence::create(  
            MoveBy::create(0.7f, Point(0, 30)),  
            CallFunc::create(CC_CALLBACK_0(Tauren::bloodTipFlyEnd, this, label)),  
            nullptr
```

```

    )
);

this->addChild(label, 15);
}

```

在 Level2Scene 类中添加击退技能 CD 和击退技能按钮，并绑定回调函数等，不再赘述。
Level2Scene 类增加一个存当前场景中的所有子弹的 vector。

```
std::vector<Bullet*> knockbackBullets;
```

Level2Scene 类声明并实现了判断子弹是否飞出屏幕的函数 isBulletOutOfScreen()、判断子弹是否击中 Tauren 的函数 isBulletHitTauren()、处理子弹命中 Tauren 的函数 workBulletHitTauren()和更新子弹的函数 updateBullets()。值得注意的是，子弹命中 Tauren 后不直接删除，而是将其移到屏幕外，待下次遍历 knockbackBullets 时才删除，防止在遍历过程中删除 vector 中的元素导致迭代器失效。

```

// 判断子弹是否飞出屏幕
bool Level2Scene::isBulletOutOfScreen(Bullet* bullet) {
    float bulletX = bullet->getBullet()->getPositionX(), bulletY =
bullet->getBullet()->getPositionY();
    return (bulletX < 0) || (bulletX > visibleSize.width) || (bulletY < 0) || (bulletY >
visibleSize.height);
}

// 判断子弹是否击中 Tauren
bool Level2Scene::isBulletHitTauren(Bullet* bullet) {
    float bulletX = bullet->getBullet()->getPositionX();
    float taurenX = tauren->getPositionX();
    return taurenX - taurenWidth / 2 < bulletX && bulletX < taurenX + taurenWidth / 2;
}

// 处理子弹命中 Tauren
void Level2Scene::workBulletHitTauren(Bullet* bullet) {
    tauren->playHurt(bullet->getDamage());

    // 击退
    if (bullet->getKnockback()) {
        float targetX = tauren->getPositionX() + bullet->getSpeed().x * BULLET_FORCE;
        targetX = std::max(targetX, taurenWidth), targetX = std::min(targetX, visibleSize.width
- taurenWidth / 2);
        tauren->setPositionX(targetX);
    }

    // 此处不直接删除，而是将其移到屏幕外，防止一个指针被多次删除
    bullet->getBullet()->setPositionX(2 * visibleSize.width);
}

```

```

// 更新子弹
void Level2Scene::updateBullets() {
    std::vector<Bullet*> trash;

    // 击退的子弹
    for (auto& bullet : knockbackBullets) {
        // 飞出屏幕
        if (isBulletOutOfScreen(bullet)) {
            trash.push_back(bullet);
            continue;
        }

        // 与 Tauren 的碰撞检测
        if (isBulletHitTauren(bullet)) {
            workBulletHitTauren(bullet);
        }
    }
    for (auto& knockbackBullet : trash) {
        knockbackBullet->removeBullet();
        knockbackBullets.erase(find(knockbackBullets.begin(), knockbackBullets.end(),
knockbackBullet));
    }
}

```

击退技能按钮如图 6.7.2.1 所示，子弹如图 6.7.2.2 所示。



图 6.7.2.1: 击退技能按钮



图 6.7.2.2: 子弹之一

6.7.3 Tauren 技能 – 隐身

Tauren 的技能 – 隐身：

- (1) CD: 500。
- (2) 效果: 5 s 内 Tauren 隐身，无法被 Hero 看到。Tauren 隐身开启护甲技能后，盾牌不会隐身。

在 Config.h 中指定隐身技能的 CD 和 Tauren 的隐身时间。

```

// Tauren 技能 CD
...
static const int TAUREN_STEALTH_CD = 500;

// 参数
...

```

```
static const int TAUREN_STEALTH_TIME = 5; // Tauren 隐身时间 (单位: s)
```

在 TaurenAI 类的 updateSkills()函数中添加隐身的淡入淡出效果。

```
// 更新所有技能
void TaurenAI::updateSkills() {
    ...

    if (stealthCD) {
        stealthCD++;
        if (stealthCD >= TAUREN_STEALTH_CD) {
            auto action = Sequence::create(
                FadeOut::create(0.5f),
                DelayTime::create(4.0f),
                FadeIn::create(0.5f),
                nullptr
            );
            tauren->getArmature()->runAction(action);

            stealthCD = 1;
        }
    }
}
```

Tauren 隐身过程如图 6.7.3.1 所示，隐身结果如图 6.7.3.2 所示。



图 6.7.3.1: Tauren 正在隐身



图 6.7.3.2: Tauren 隐身

6.8 第三关

《侠客行》第三关的参数：

关卡	Hero				Tauren			
	平 A	防御	击退	弹幕	平 A	护甲	隐身	击飞
三	✓	✓	✓	✓	✓	✓	✓	✓

Hero 血量：100。

Tauren 血量：200。

Tauren AI 的决策时间：200。

6.8.1 Hero 大招 – 弹幕

Hero 的大招 – 弹幕：

(1) CD：500。

(2) 效果：在 Hero 上方出现一个自旋的弹幕发射器，每隔一段时间会向四周发射子弹，子弹飞行过程中有 80%概率会有随机偏移。弹幕发射器会随 Hero 移动。

先在 Config.h 中指定弹幕技能的 CD、贴图路径和参数。

```

// HERO 技能 CD
...
static const int BARRAGE_CD = 500;

// 技能贴图
...
static const std::string BARRAGE_EMITTER_PATH = "SkillItems/emitter.png";

// 参数
...
static const int BARRAGE_EMITTER_ROTATE_SPEED = 60; // 弹幕发射器旋转速度 (0.1 s
相差的角度)
static const int HERO_BARRAGE_TIME = 300; // 弹幕持续时间
static const int BARRAGE_DELTA_TIME = 50; // 弹幕的发射时间间隔
static const int BARRAGE_DELTA_ANGLE = 30; // 相邻弹幕的发射角度差
static const int BARRAGE_BASIC_DAMAGE = 1; // 弹幕的基础伤害

```

Level3Scene 类新增成员变量：弹幕计时器 barrageTimer、弹幕发射器 barrageEmitter 和存场景中所有弹幕的子弹的 barrageBullets。

```

int barrageTimer; // 弹幕计时器
Sprite* barrageEmitter; // 弹幕发射器

...

std::vector<Bullet*> barrageBullets;

```

在 updateBullets()函数中加入弹幕的子弹的处理。

```

// 更新子弹
void Level3Scene::updateBullets() {
    ...

    // 弹幕的子弹
    for (auto& bullet : barrageBullets) {
        // 飞出屏幕
        if (isBulletOutOfScreen(bullet)) {
            trash.push_back(bullet);
            continue;
        }

        // 与 Tauren 的碰撞检测
        if (isBulletHitTauren(bullet)) {
            workBulletHitTauren(bullet);
        }
    }
    for (auto& barrageBullet : trash) {
        barrageBullet->removeBullet();
    }
}

```



```

        auto it = find(barrageBullets.begin(), barrageBullets.end(), barrageBullet);
        if (it != barrageBullets.end()) {
            barrageBullets.erase(it);
        }
    }
}

```

定义并实现加载发射器的函数 `loadBarrageEmitter()`，为发射器添加自旋效果。初始时发射器不可见。

```

// 加载弹幕发射器
void Level3Scene::loadBarrageEmitter() {
    barrageEmitter = Sprite::create(BARRAGE_EMITTER_PATH);
    barrageEmitter->setPositionY(300);
    barrageEmitter->setVisible(false);

    // 旋转动作
    auto actionSequence = Sequence::create(
        RotateBy::create(0.1f, BARRAGE_EMITTER_ROTATE_SPEED),
        nullptr
    );
    auto action = RepeatForever::create(actionSequence);
    barrageEmitter->runAction(action);

    this->addChild(barrageEmitter, 15);
}

```

定义并实现更新弹幕的函数 `updateBarrage()`，它更新弹幕发射器的可见性、弹幕计时器，随机给已有的弹幕加随机偏移，并每隔一段时间朝四周发射弹幕。新增弹幕时，枚举角度，在各个角度新增对应初速度方向的子弹即可。

```

// 更新弹幕
void Level3Scene::updateBarrage() {
    if (!barrageTimer) {
        barrageEmitter->setVisible(false);

        return;
    }

    // 更新发射器的位置和可视性
    barrageEmitter->setPositionX(hero->getPositionX());
    barrageEmitter->setVisible(true);

    // 更新弹幕计时器
    barrageTimer++;
    if (barrageTimer >= HERO_BARRAGE_TIME) {
        barrageTimer = 0;
    }
}

```

```

// 更新弹幕
for (auto& barrageBullet : barrageBullets) {
    // 80% 概率子弹偏移
    int shiftingRand = generateRandomInteger(1, 10);
    if (shiftingRand <= 2) {
        continue;
    }

    // 随机偏移向量
    Vec2 shiftingSpeed = Vec2(generateRandomInteger(-2, 2), generateRandomInteger(-2,
2));

    // shiftingSpeed = shiftingSpeed.getNormalized();

    float speedLength = barrageBullet->getSpeed().getLength();
    barrageBullet->setSpeed(barrageBullet->getSpeed() + shiftingSpeed);
    barrageBullet->normalizeSpeed(speedLength);
}

// 新增弹幕
if (barrageTimer % BARRAGE_DELTA_TIME) {
    return;
}

for (int angle = 0; angle <= 360; angle += BARRAGE_DELTA_ANGLE) {
    float radian = angle * pi / 180;
    Bullet* bullet = Bullet::create(
        barrageEmitter->getPosition(),
        Vec2(BULLET_SPEED * cos(radian), BULLET_SPEED * sin(radian)),
        BARRAGE_BASIC_DAMAGE,
        false
    );
    this->addChild(bullet, 20);

    barrageBullets.push_back(bullet);
}
}

```

弹幕技能的运行结果如图 6.8.1.1 所示。

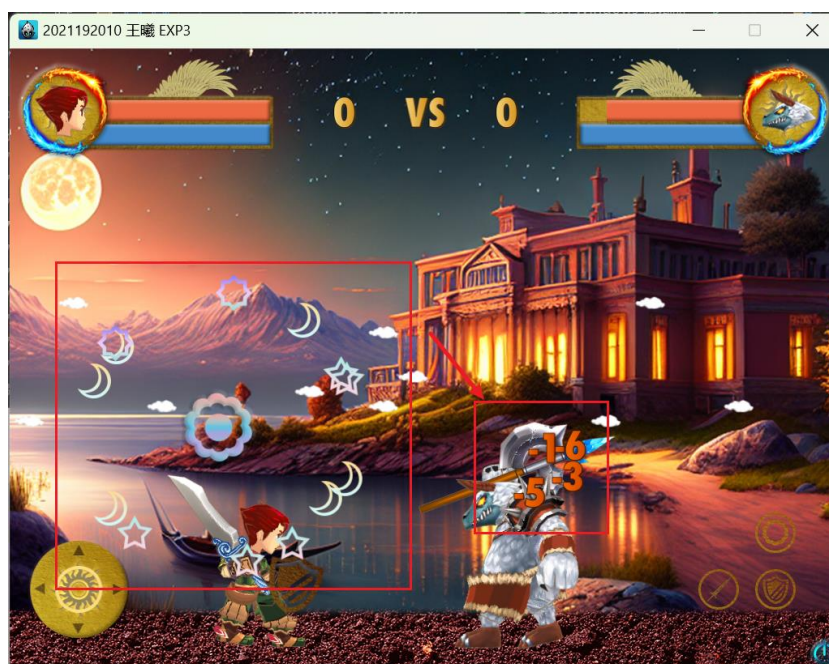


图 6.8.1.1: 弹幕技能

6.8.2 Tauren 大招 – 击飞

Tauren 的大招 – 击飞:

(1) CD: 600。

(2) 效果: Tauren 进行一次攻击, Hero 不管在何处都会被击飞。Tauren 攻击时, Hero 离 Tauren 越近, 被击飞的高度越高。Hero 被击飞后做竖直上抛运动, 落地扣血, 高度越高, 扣血越多。

先在 Config.h 中指定击飞技能的 CD 和参数。

```
// Tauren 技能 CD
...
static const int TAUREN_STRIKE_FLY_CD = 600;

// 参数
...
static const float STRUCK_FLY_BASIC_SPEED = 50; // 击飞的初速度 (最大)
static const float g = 0.98; // 重力加速度
static const int STRUCK_FLY_BASIC_DAMAGE = 30; // 击飞的基础伤害
```

Hero 类新增击飞初速度变量 `struckFlySpeed` 和显示击飞落地扣血的函数 `displayBloodTip()`, 其中扣血恒为暴击。

```
// 显示扣血 (击飞)
void Hero::displayBloodTip(int basicDamage) {
    int damage = basicDamage + generateRandomInteger(0, 10);
    setHP(HP - damage);

    auto label = Label::createWithBMFont(TIPS_FNT_PATH, StringUtils::format("-%d",
damage));
    label->setColor(Color3B::RED);
    label->setPosition(Point(20 + generateRandomInteger(0, 80), 10 +
```

```

generateRandomInteger(0, 80));
    label->runAction(
        Sequence::create(
            MoveBy::create(0.7f, Point(0, 30)),
            CallFunc::create(CC_CALLBACK_0(Hero::bloodTipFlyEnd, this, label)),
            nullptr
        )
    );

    this->addChild(label, 15);
}

```

TaurenAI 类新增击飞技能的 CD，并在 updateSkills() 函数中加入击飞的实现。具体地，根据 Hero 与 Tauren 的距离占屏幕宽度的比例，给予 Hero 击飞的初速度。

```

// 更新所有技能
void TaurenAI::updateSkills() {
    ...

    if (strikeFlyCD) {
        strikeFlyCD++;
        if (strikeFlyCD >= TAUREN_STRIKE_FLY_CD) {
            tauren->getArmature()->getAnimation()->play("attack");

            // Hero 离 Tauren 越近，击飞的初速度越大
            float distance = fabs(hero->getPositionX() - tauren->getPositionX());
            hero->setStruckFlySpeed(STRUCK_FLY_BASIC_SPEED * (1 - distance /
Director::getInstance()->getVisibleSize().width));

            strikeFlyCD = 1;
        }
    }
}

```

Hero 被击飞后做竖直上抛运动，速度变化量为 $g = 0.98$ 。Level3Scene 类新增用于实现竖直上抛的成员变量：击飞 y 速度 struckFlySpeed 和击飞是否进入下落阶段 struckFlyDown。

```

float struckFlySpeed; // 击飞 y 速度
bool struckFlyDown; // 击飞进入下落阶段

```

在 update() 函数中加入更新 Hero 被击飞的函数 updateHeroBeStruckFly()，它分为初始化击飞速度、上升阶段和下落阶段。下落结束后，玩家扣血。此外，下落结束后将 Hero 的 y 置为初始值，防止下落速度过大导致错位。

```

// 更新 Hero 被击飞
void Level3Scene::updateHeroBeStruckFly() {
    if (!doubleCompare(hero->getStruckFlySpeed(), 0)) {
        return;
    }
}

```

```

// 上升初速度
if (!doubleCompare(struckFlySpeed, 0) && !struckFlyDown) {
    struckFlySpeed = hero->getStruckFlySpeed();
}

if (struckFlyDown) { // 下落
    hero->setPositionY(hero->getPositionY() - struckFlySpeed);
    struckFlySpeed += g;

    // 下落结束
    if (doubleCompare(hero->getPositionY(), 30) <= 0) {
        hero->setPositionY(30);

        hero->setStruckFlySpeed(0);

        struckFlySpeed = 0;
        struckFlyDown = false;

        hero->displayBloodTip(STRUCK_FLY_BASIC_DAMAGE);
    }
}
else { // 上升
    hero->setPositionY(hero->getPositionY() + struckFlySpeed);
    struckFlySpeed -= g;
    if (doubleCompare(struckFlySpeed, 0) <= 0) {
        struckFlyDown = true;
    }
}
}

```

运行结果如图 6.8.2.1 所示，Hero 被击飞。

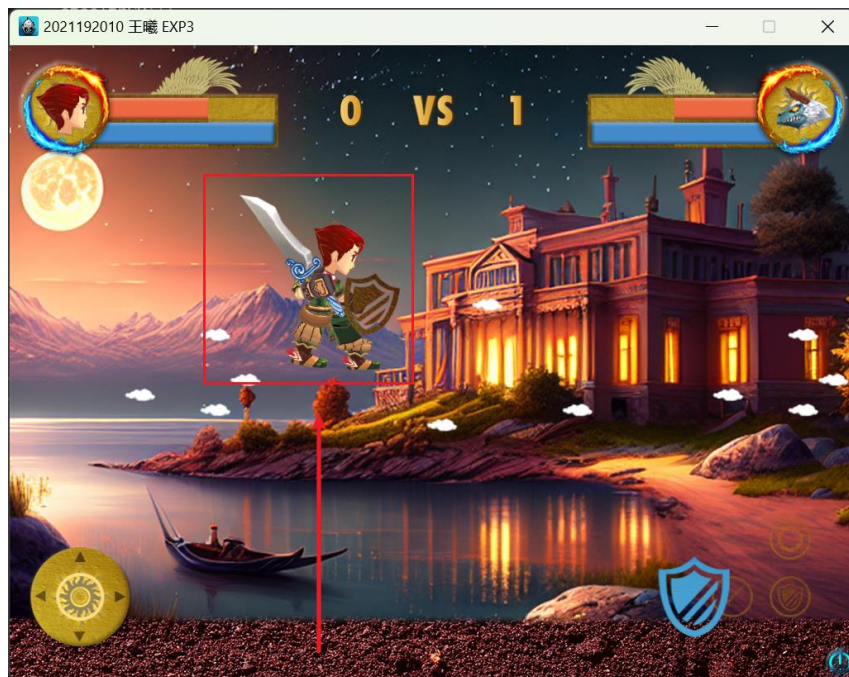


图 6.8.2.1: Hero 被击飞

6.9 通关界面

实现与 6.4 开始界面基本相同，不再赘述。

运行结果如图 6.9.1 所示。

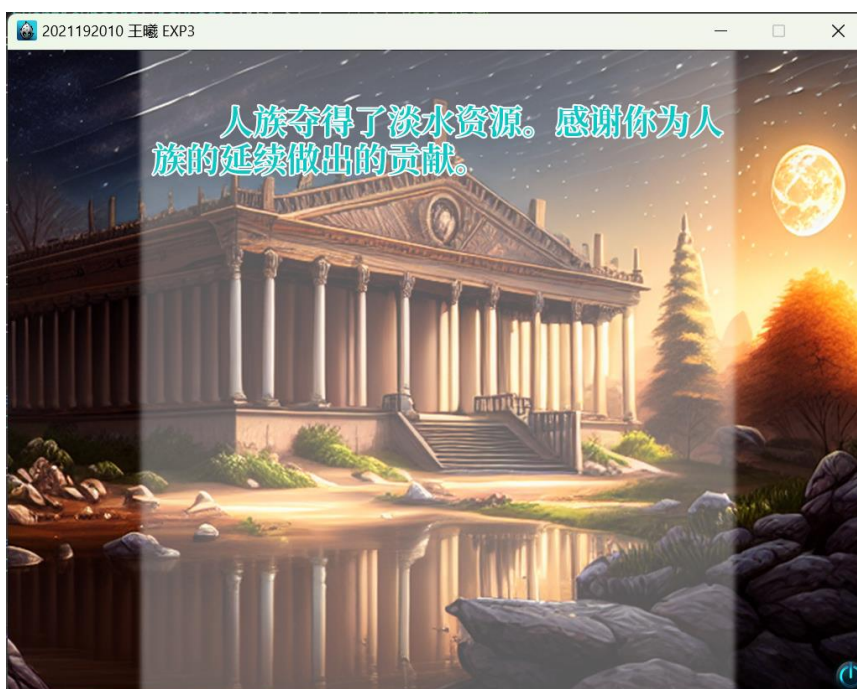


图 6.9.1: 通关界面

四、实验结论或心得体会

7. 实验结论

运行录屏见【附件】侠客行。

7.1 编译运行

成功编译并运行《侠客行》。如图 7.1.1 所示。

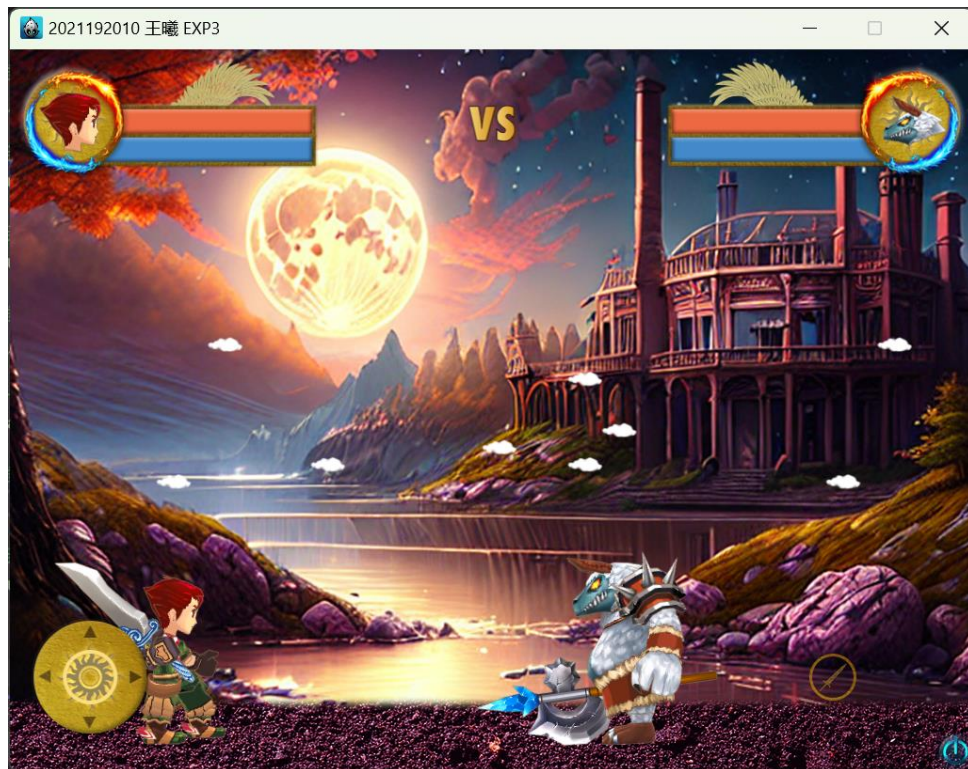


图 7.1.1：成功编译并运行《侠客行》

7.2 增加计分板与多重关卡

7.2.1 计分板

以 Hero 打败 Tauren 为例，运行结果如图 7.2.1 所示，Hero 得 1 分。

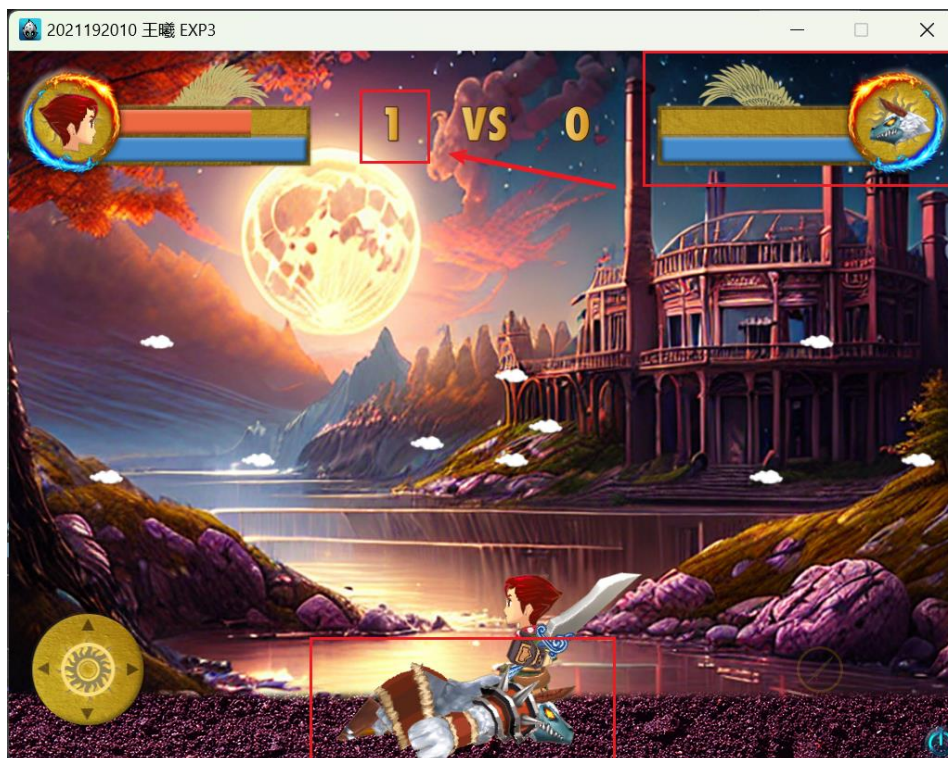


图 7.2.1: Hero 得 1 分

7.2.2 多重关卡

《侠客行》设 3 个关卡，丰富度、难度递增，主要体现在技能上。具体地：

关卡	Hero				Tauren			
	平 A	防御	击退	弹幕	平 A	护甲	隐身	击飞
一	✓	✓			✓	✓		
二	✓	✓	✓		✓	✓	✓	
三	✓	✓	✓	✓	✓	✓	✓	✓

此外，3 个关卡的 Tauren AI 会越来越“聪明”，表现为根据 Hero 与 Tauren 的相对位置做决策的时间缩短。具体地：

- (1) 第一关：决策时间 400。
- (2) 第二关：决策时间 300。
- (3) 第三关：决策时间 200。

3 个关卡的概览如图 7.2.2.1、图 7.2.2.2 和图 7.2.2.3 所示。



图 7.2.2.1: 第一关



图 7.2.2.2: 第二关

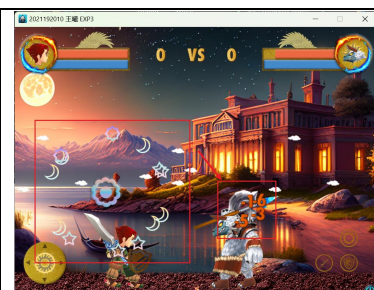


图 7.2.2.: 第三关

7.3 增加 Hero 的 Defend 动作

成功用 Cocos Studio 的 Animation Editor 绘制 Hero 的 Defend 动作。如图 7.3.1 所示。

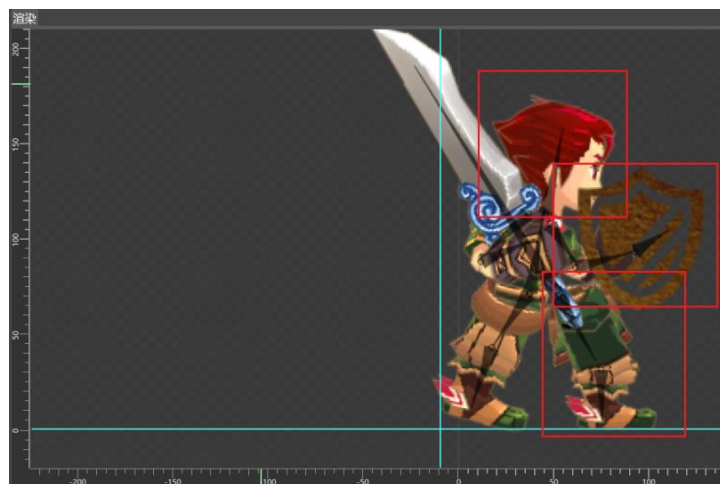


图 7.3.1: Defend 动作

成功在场景中加载，有无防御受伤时的掉血如图 7.3.2 和图 7.3.3 所示。



图 7.3.2: 无防御受伤



图 7.3.3: 有防御受伤

7.4 修复 Bug

修复了如下 Bug:

- (1) Hero 受伤后无法操控
- (2) 攻击按钮也控制了 Tauren
- (3) 多次掉血累加
- (4) Tauren 左倾
- (5) 角色的移动超出世界边界
- (6) Tauren 的攻击区域过小
- (7) 碰撞检测不严谨
- (8) 连续点击技能
- (9) smitten 动画未执行
- (10) attack 动画会被 smitten 动画打断
- (11) 云朵移动动画衔接不上
- (12) 角色朝向切换
- (13) Hero 诈尸
- (14) 攻击、受伤后无法恢复 loading 动作
- (15) Hero 很靠近 Tauren 时持续受伤
- (16) 角色受伤时攻击被打断

7.5 增加配乐和音效

本次实验选择《妖精的尾巴》的 3 首 OP 分别作为 3 关的 bgm。

- (1) 第一关：高梨康治 - 魔法発動
- (2) 第二关：高梨康治 - Dokuryuu No Cobra
- (3) 第三关：高梨康治 - FAIRY TAIL メインテーマ

7.6 游戏优化

7.6.1 剧情

加入游戏背景、通关剧情。

7.6.2 Tauren AI 优化

优化了 Tauren AI，使得 Tauren 会根据它与 Hero 的相对位置决定倾向往哪边走，且可调节 Tauren AI 的灵敏度。此外，Tauren AI 还实现 Tauren 定期释放技能。

7.6.3 随机性优化

为增强游戏随机性做了如下优化：

- (1) 攻击有 20%的概率二连击，有 5%的概率三连击。
- (2) 攻击有 20%的概率暴击。
- (3) 攻击有 20%的概率 Miss。

上述概率相互独立。

7.6.4 技能设置

Hero 技能：

- (1) 平 A：普通近战攻击。
- (2) 防御：举盾期间抵消部分伤害。
- (3) 击退：远程攻击，可击退敌人。
- (4) 【大招】弹幕：Hero 所在位置上方出现弹幕发射器，往四周发射弹幕，弹幕会对敌人造成伤害。

Tauren 技能：

- (1) 平 A：普通近战攻击。
- (2) 护甲：出现一个可以抵挡一次攻击的护甲。
- (3) 隐身：一段时间内无法被敌人看见。
- (4) 【大招】击飞：全屏击飞，敌人离 Tauren 越近伤害越高，敌人落地掉血。

8. 实验心得

(1) 开始实验前将样例代码重构一遍 helps a lot, 不仅是因为理顺了实现思路, 也因为样例代码的实现太丑了。

(2) 花时间最多的部分: 测试和 Debug。

(3) 有时修好一个 Bug 会带来新的 Bug, 所以写代码的本质是在写 Bug。

(4) 做游戏真的是肝上长了个人, 除 AppDelegate.h 和 AppDelegate.cpp 外的代码行数如下。

\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\BasicScene.cpp	226
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\BasicScene.h	90
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\BloodBar.cpp	95
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\BloodBar.h	46
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Bullet.cpp	93
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Bullet.h	46
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\CollisionDetection.cpp	126
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\CollisionDetection.h	42
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Config.cpp	15
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Config.h	138
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\EndScene.cpp	113
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\EndScene.h	32
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Headers.h	25
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\HelloWorldScene.cpp	133
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\HelloWorldScene.h	44
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Hero.cpp	366
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Hero.h	93
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Joystick.cpp	109

\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Joystick.h	45
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level1Scene.cpp	527
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level1Scene.h	141
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level2Scene.cpp	644
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level2Scene.h	162
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level3Scene.cpp	835
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Level3Scene.h	183
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\StartScene.cpp	113
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\StartScene.h	33
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Tauren.cpp	337
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\Tauren.h	87
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\TaurenAI.cpp	227
\\计算机游戏开发\\实验\\实验 3 二维游戏动画合成\\code\\Classes\\TaurenAI.h	86

合计如图 8.1 所示。

```

C:\Users\Hytidel>python3
Python 3.11.7 (main, Dec 7 2023, 09:09:57) [GCC UCRT 13.2.0 64 bit (AMD64)] on w
in32
Type "help", "copyright", "credits" or "license" for more information.
>>> 226 + 90 + 95 + 46 + 93 + 46 + 126 + 42 + 15 + 138 + 113 + 32 + 25 + 133 + 44
+ 366 + 93 + 109 + 45 + 527 + 141 + 644 + 162 + 835 + 183 + 113 + 33 + 337 + 87 +
227 + 86
5252
>>>

```

图 8.1: 项目代码行数

指导教师批阅意见：	
成绩评定： 评语：	
	指导教师签字：_____ 年 月 日
备注：	

评语:

指导教师签字:

年 月 日

备注:

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。